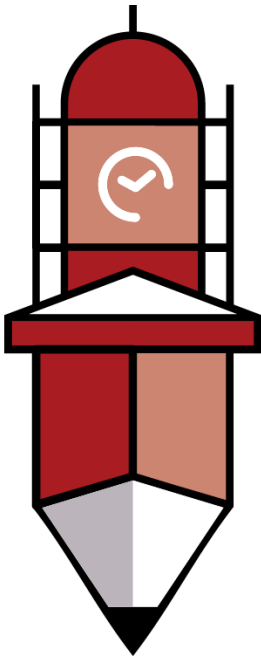


DATA MINING (DM)



The University of Faisalabad

SUBMIT BY: AHTISHAM AKRAM.

SUBMIT TO: SIR, ARHAM

REG NO: 2023-BS-AI-166

DEPARTMENT OF COMPUTATIONAL SCIENCES

FACULTY OF INFORMATION TECHNOLOGY

The University of Faisalabad

2026

Lab	Topic	Tools
1	Introduction	KNIME Analytics Platform
2	CRISP-DM	Orange Data Mining
3	Data Cleaning	Python + RapidMiner
4	Data Preprocessing	Python + RapidMiner
5	Market Basket Analysis	Python (Manual + mlxtend)
6	Apriori Algorithm	Python + KNIME
7	FP-Growth Algorithm	Python + Orange
8	Python Basics / OOP	Python
9	Decision Tree	KNIME + Python
10	Naïve Bayes & KNN	RapidMiner + Python
11	SVM	KNIME + Python
12	Clustering	Orange + Python
13	Outlier Detection	RapidMiner + Python
14	Scraping & Sentiment	Citespace / Python
15	Final Capstone	All Tools

LAB 1	Introduction to Data Mining
------------------	------------------------------------

Topic	Introduction to Data Mining
Problem	Retail store wants sales insights
Dataset	Retail Sales Dataset (Supermarket Sales – Kaggle)
Tools	KNIME Analytics Platform
KPIs	Total Revenue, Avg Order Value, Category Revenue, Top Products

Learning Objectives

- Import dataset and identify data types
- Generate summary statistics for key fields
- Explore attributes and understand distributions
- Create basic visual reports in KNIME

Background

Data Mining is the process of discovering patterns, correlations, anomalies, and insights from large datasets. In this introductory lab, you will work with a retail supermarket sales dataset to understand how raw transactional data can be transformed into business intelligence.

KNIME Analytics Platform is an open-source data analytics, reporting, and integration platform that allows building workflows through a visual node-based interface — no programming required for basic tasks.

Dataset Description

Column	Data Type	Description
Invoice ID	String	Unique transaction identifier
Branch	String	Store branch (A, B, C)
City	String	City where branch is located
Customer type	String	Member or Normal
Gender	String	Customer gender
Product line	String	Product category
Unit price	Float	Price per item (\$)
Quantity	Integer	Units purchased
Total	Float	Total transaction amount
Date	Date	Transaction date
Payment	String	Payment method
Rating	Float	Customer satisfaction (1-10)

Lab Procedure – KNIME Workflow

STEP 1 Import Dataset

Open KNIME Analytics Platform. Create a new workflow named "Lab1_Retail_Sales". Add a CSV Reader node and configure it to load the Supermarket Sales dataset.

- File → Import KNIME Workflow or drag CSV Reader from Node Repository
- Double-click the node to configure the file path
- Execute node (Right-click → Execute) and preview data

STEP 2 Identify Data Types

Add a Statistics node connected to the CSV Reader output.

- The Statistics node shows: count, mean, std, min, max for numeric columns
- Check column types using Column Filter or Data Explorer node
- Ensure numeric columns are not imported as strings

STEP 3 Generate Summary Statistics

Connect a Statistics node and a GroupBy node to compute revenue by category.

- Statistics node: inspect Total, Unit price, Quantity, Rating
- GroupBy node: group by "Product line", aggregate SUM(Total), COUNT(Invoice ID)
- Add a Sorter node to rank product lines by total revenue

STEP 4 Visualize with Bar Chart

Add a Bar Chart node from the Views section to create a visual of Category Revenue.

- X-axis: Product line | Y-axis: Sum(Total)
- Execute and open the interactive view
- Export the chart as PNG using the Image Writer node

KPI Calculations

KPI	Formula	Expected Insight
Total Revenue	SUM(Total)	Overall store performance
Avg Order Value	SUM(Total) / COUNT(Invoice ID)	Customer spending behavior
Category Revenue	SUM(Total) GROUP BY Product line	Best-selling product category
Top 5 Products	Sort by Total DESC, LIMIT 5	High-value items to promote

Expected Results

- Total transactions: ~1,000 rows

- Top revenue category: Typically "Food and beverages" or "Electronic accessories"
- Average order value: ~\$300-\$350
- Customer rating distribution: Mostly between 5.5 – 8.5

Lab Tasks

1. Load the dataset and report the total number of rows and columns.
2. Identify which columns contain missing values (if any).
3. Calculate total revenue for each product line.
4. Find the top 5 invoices by total amount.
5. Create a bar chart of product line vs. total revenue. Export and attach to your report.
6. Which branch (A/B/C) has the highest average customer rating?

The screenshot displays the Alteryx interface with a data workflow and a k-Means cluster analysis panel.

Workflow:

- CSV Reader:** Loads the dataset.
- GroupBy:** Groups data by Product_ID.
- Sorter:** Sorts data by Sales_Amount (descending).
- k-Means:** Performs k-Means clustering on the data.

k-Means Panel:

- Column selection:** Manual, Wildcard, Regex, Type.
- Search:** Search bar.
- Excludes:** No columns in this list.
- Includes:** Product_ID, Sales_Amount, Quantity_Sold, Unit_Cost, Unit_Price, Discount.
- Number of clusters:** 3.
- Centroid initialization:** Discard, Apply and Execute, Apply.

Table 1: Labeled input

#	RowID	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity	Product_Line	Unit_Cost	Unit_Price	Customer_Rating	Discount	Payment_Method	Sales_Channel
1	Row0	1052	2023-02-03	Bob	North	5,053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online
2	Row1	1093	2023-04-21	Bob	West	4,384.02	17	Furniture	3,816.39	4,209.44	Returning	0.11	Cash	Retail
3	Row2	1015	2023-09-21	David	South	4,631.23	30	Food	261.56	371.4	Returning	0.2	Bank Transfer	Retail
4	Row3	1072	2023-08-24	Bob	South	2,167.94	39	Clothing	4,330.03	4,467.75	New	0.02	Credit Card	Retail
5	Row4	1061	2023-03-24	Charlie	East	3,750.2	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online
6	Row5	1021	2023-02-11	Charlie	West	3,761.15	32	Food	900.79	1,106.51	New	0.21	Cash	Online
7	Row6	1083	2023-04-11	Bob	West	618.31	29	Furniture	2,408.81	2,624.09	Returning	0.14	Cash	Online
8	Row7	1087	2023-01-06	Eve	South	7,698.92	46	Furniture	3,702.51	3,964.65	New	0.12	Bank Transfer	Online
9	Row8	1075	2023-06-29	David	South	4,223.39	30	Furniture	738.06	1,095.45	New	0.05	Bank Transfer	Online

Table 1: Group table

#	RowID	Max*(Sales_Amount)	Min*(Sales_Amount)
1	Row0	9,989.04	100.12

CSV Reader

k-Means

GroupBy

GroupBy

Sorter

Add comment

Sorter

100%

Sorter

Sorting

Criterion 1

Column

Min*(Sales_Amount)

Order

Ascending

Descending

Add sorting criterion

Show advanced settings

Discard

Apply and Execute

Apply

1: Sorted Table

Flow Variables

Table

Statistics

Rows: 1

Columns: 2

#	RowID	Max*(Sales_Amount) Number (Float)	Min*(Sales_Amount) Number (Float)
1	Row0	9,989.04	100.12

LAB 2

CRISP-DM Methodology

Topic	CRISP-DM Process Model
Problem	E-commerce company wants customer churn prediction
Dataset	Customer Churn Dataset (Kaggle)
Tools	Orange Data Mining
KPIs	Churn Rate, Retention Rate, Accuracy, ROC-AUC

Learning Objectives

- Understand the six phases of the CRISP-DM lifecycle
- Apply each phase to a real-world churn prediction problem
- Build a basic predictive model using Orange Data Mining
- Evaluate model performance using standard metrics

CRISP-DM Overview

CRISP-DM (Cross-Industry Standard Process for Data Mining) is the most widely used methodology for data mining projects. It provides a structured, iterative approach with six phases:

Phase	Description	Key Outputs
1. Business Understanding	Define objectives and success criteria	Project charter, KPI definition
2. Data Understanding	Collect and explore initial data	EDA report, data quality notes
3. Data Preparation	Clean, transform, and engineer features	Analysis-ready dataset
4. Modeling	Select and apply modeling techniques	Trained model(s)
5. Evaluation	Assess results against business objectives	Performance metrics, ROC curve
6. Deployment	Deploy model to production environment	Deployed pipeline, documentation

Phase 1 – Business Understanding

Business Goal: Predict which customers are likely to cancel their subscription (churn) so the company can proactively offer retention incentives.

Success Criteria: Model accuracy $\geq 80\%$, ROC-AUC ≥ 0.75

Constraints: Must be explainable to non-technical stakeholders

Phase 2 – Data Understanding

The Customer Churn dataset typically contains:

- CustomerID: Unique identifier
- Tenure: Number of months as customer
- MonthlyCharges: Monthly billing amount
- TotalCharges: Cumulative billing
- Contract: Month-to-month, One year, Two year
- Churn: Target variable (Yes / No)

Phase 3 – Data Preparation (Orange Workflow)

STEP 1 Load Data in Orange

- Drag a File widget onto the canvas and load the churn CSV
- Connect to a Data Table widget to inspect the data

STEP 2 Handle Missing Values

- Add an Impute widget — use median for numeric, most frequent for categorical
- Verify in Data Info widget: 0 missing values after imputation

STEP 3 Feature Selection

- Use Select Columns widget to remove CustomerID (non-predictive)
- Ensure Churn is set as the target variable (class)

Phase 4 – Modeling

STEP 4 Build Classification Model

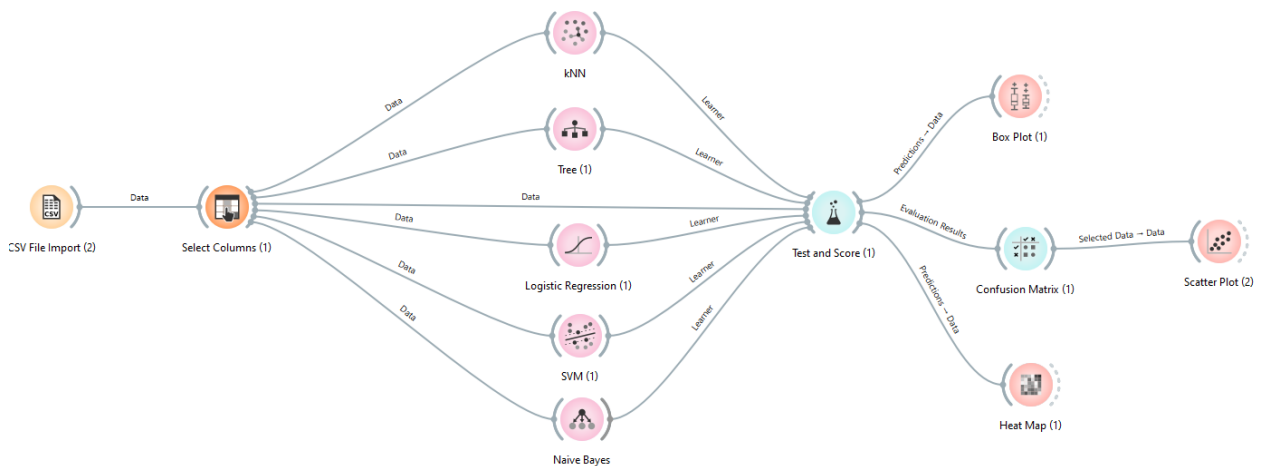
- Add a Logistic Regression learner and a Naive Bayes learner
- Connect both to a Test and Score widget
- Use 10-fold cross-validation for evaluation

Phase 5 – Evaluation

Metric	Formula	Target
Accuracy	$(TP + TN) / \text{Total}$	$\geq 80\%$
Churn Rate	Churned / Total Customers	Identify high-risk segment
Retention Rate	$1 - \text{Churn Rate}$	Track improvement over time
ROC-AUC	Area Under ROC Curve	≥ 0.75

Lab Tasks

- Map the churn prediction problem to all 6 CRISP-DM phases. Write 2-3 sentences per phase.
- Load the dataset in Orange. Report the churn rate (% of churned customers).
- Apply Impute to handle missing values. Before vs. after: how many missing cells?
- Train a Logistic Regression model. Report Accuracy and ROC-AUC.
- Compare two models (e.g., Logistic Regression vs. Naive Bayes). Which performs better?
- Which feature has the strongest correlation with churn? Justify your answer.



Test and Score (1) - Orange

☐ Cross validation
 Number of folds: 5
☒ Stratified

☐ Cross validation by feature

☒ Random sampling
 Repeat train/test: 20
 Training set size: 80 %
☒ Stratified

☐ Leave one out
☐ Test on train data
☐ Test on test data

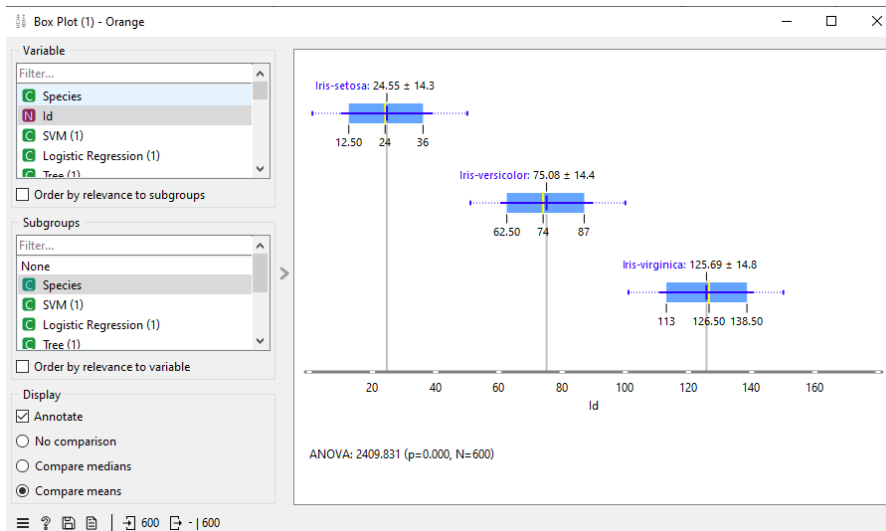
Evaluation results for target: (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
SVM (1)	0.997	0.960	0.960	0.960	0.960	0.940
Logistic Regression (1)	0.997	0.963	0.963	0.964	0.963	0.946
Tree (1)	0.962	0.933	0.933	0.933	0.933	0.900
kNN	0.988	0.963	0.963	0.964	0.963	0.946
Naive Bayes	0.984	0.893	0.893	0.894	0.893	0.841

Compare models by: Area under ROC curve ☐ Negligible diff.: 0.1

	SVM (1)	Logistic Re...	Tree (1)	kNN	Naive Bayes
SVM (1)					
Logistic Regression (1)					
Tree (1)					
kNN					
Naive Bayes					

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.



Confusion Matrix (1) - Orange

Learners

- SVM (1)
- Logistic Regression (1)
- Tree (1)
- kNN
- Naive Bayes

Output

☒ Predictions

☐ Probabilities

☒ Apply Automatically

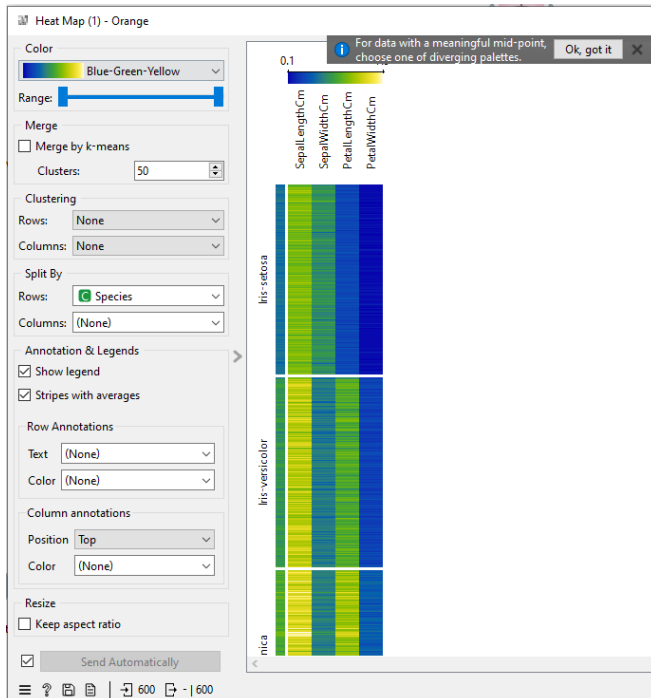
5x600 | 162 | 600

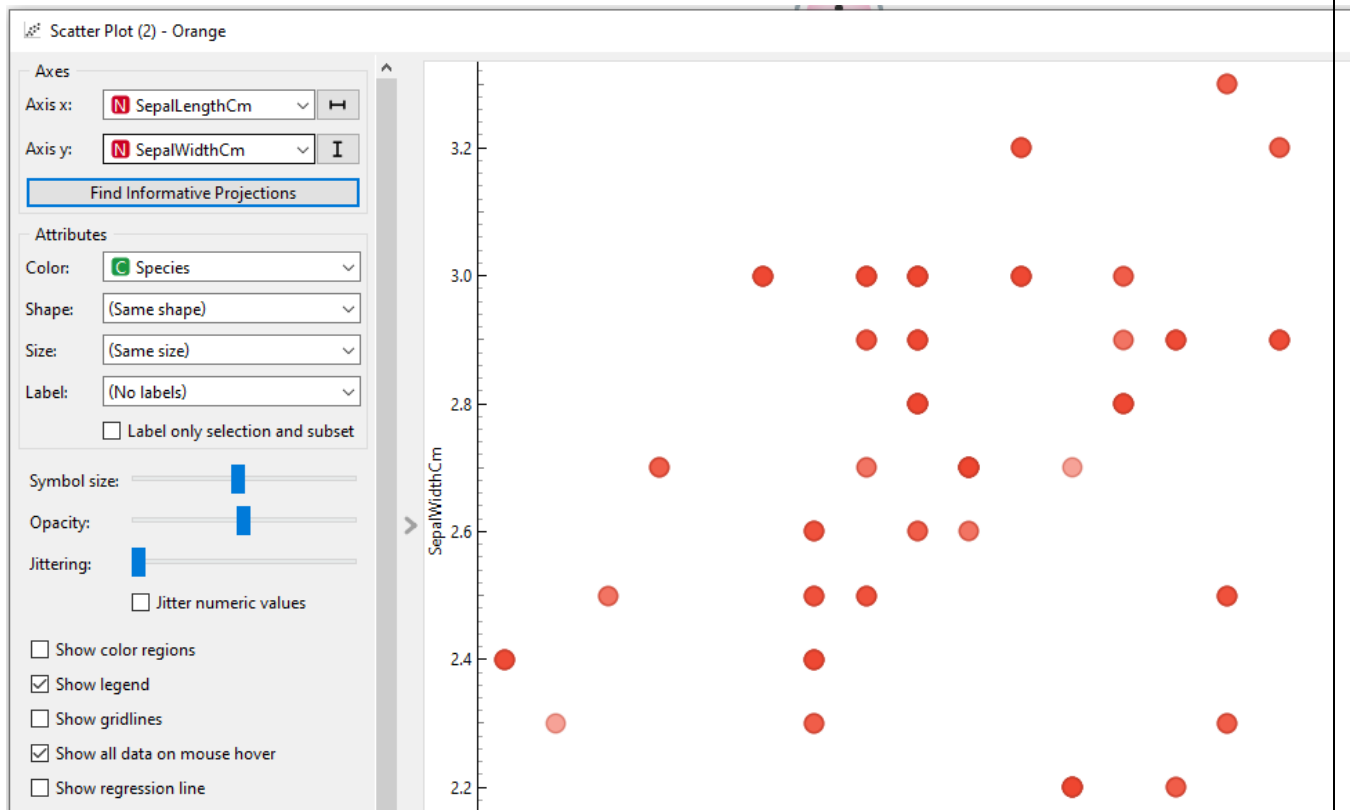
Show: Number of instances

Predicted

		Predicted			
		Iris-setosa	Iris-versicolor	Iris-virginica	Σ
Actual	Iris-setosa	199	1	0	200
	Iris-versicolor	0	162	38	200
	Iris-virginica	0	25	175	200
Σ		199	188	213	600

Select Correct Select Misclassified Clear Selection





LAB 3	Data Cleaning
------------------------	----------------------

Topic	Data Cleaning & Quality Improvement
Problem	Hospital preparing patient data for predictive modeling
Dataset	Healthcare Dataset (Synthetic)
Tools	Python (pandas, numpy, matplotlib, scipy)
KPIs	% Missing Reduced, Duplicate Removal %, Data Quality Score

Learning Objectives

- Identify and handle missing values using median and mode imputation
- Detect and remove duplicate records
- Apply domain-based rules and Z-score filtering for noise/outlier removal
- Compute and report data quality KPIs

Background

Raw healthcare data often contains errors, missing values, duplicates, and extreme outlier readings (e.g., a recorded age of 180 or blood pressure of 400). Before any predictive modeling, the data must be cleaned to ensure valid, consistent, and complete records.

Step 1 – Import Libraries

```
[1]
✓ 1s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

print('Libraries loaded successfully!')
```

Libraries loaded successfully!

Step 2 – Load / Create Dataset

```

np.random.seed(42)
n = 300

df = pd.DataFrame({
    'PatientID': range(1, n+1),
    'Age': np.random.randint(18, 90, n).astype(float),
    'Gender': np.random.choice(['Male', 'Female', None], n, p=[0.48,0.48,0.04]),
    'BloodPressure': np.random.normal(120, 15, n),
    'Cholesterol': np.random.normal(200, 40, n),
    'Glucose': np.random.normal(100, 25, n),
    'Diagnosis': np.random.choice(['Healthy', 'Diabetic', 'Hypertensive', 'Cardiac', None], n, p=[0.4,0.2,0.2,0.15,0.05])
})

# Inject missing values
for col in ['Age', 'BloodPressure', 'Cholesterol', 'Glucose']:
    idx = np.random.choice(df.index, size=int(0.08*n), replace=False)
    df.loc[idx, col] = np.nan

# Inject duplicates
df = pd.concat([df, df.sample(15, random_state=1)], ignore_index=True)

# Inject noise / outliers
df.loc[5, 'Age'] = 180
df.loc[10, 'BloodPressure'] = 400
df.loc[15, 'Glucose'] = -50

print('Dataset Shape:', df.shape)
df.head(10)

```

Dataset Shape: (315, 7)

	PatientID	Age	Gender	BloodPressure	Cholesterol	Glucose	Diagnosis
0	1	69.0	Female	125.402535	244.121011	177.621686	Healthy
1	2	32.0	Male	145.778991	NaN	NaN	Healthy
2	3	89.0	Male	132.991058	204.825889	164.839288	Diabetic
3	4	78.0	Male	118.446359	252.483678	110.305460	Hypertensive
4	5	38.0	Female	123.301115	199.670537	97.928727	Hypertensive
5	6	180.0	Male	91.741748	254.366305	90.670870	Healthy
6	7	20.0	Male	134.529427	NaN	137.743145	Diabetic
7	8	39.0	Female	129.685992	172.830323	141.012585	Healthy
8	9	70.0	Male	138.182346	272.438011	114.994596	Healthy
9	10	NaN	Male	117.839261	199.166050	128.338669	Hypertensive

Step 3 – Initial Exploration

```

print('=== Dataset Info ===')
df.info()
print()
print('=== Summary Statistics ===')
df.describe()

```

```

=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 315 entries, 0 to 314
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   PatientID             315 non-null    int64
1   Age                   290 non-null    float64
2   Gender                 302 non-null    object
3   BloodPressure          291 non-null    float64
4   Cholesterol            289 non-null    float64
5   Glucose                288 non-null    float64
6   Diagnosis              299 non-null    object
dtypes: float64(4), int64(1), object(2)
memory usage: 17.4+ KB

```

```

=== Summary Statistics ===

```

	PatientID	Age	BloodPressure	Cholesterol	Glucose
count	315.000000	290.000000	291.000000	289.000000	288.000000
mean	151.638095	54.341379	121.661842	206.321620	99.898731
std	86.242118	22.269131	23.166674	41.906582	28.737396
min	1.000000	18.000000	78.230465	100.748201	-50.000000
25%	77.500000	36.000000	108.925635	177.044421	81.169630
50%	151.000000	54.000000	121.140721	203.186314	100.803911
75%	225.500000	71.750000	132.362994	231.665966	119.327680
max	300.000000	180.000000	400.000000	326.694867	177.621686

Step 4 – Missing Value Analysis

```

missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100
total_missing_before = missing.sum()

print('=== Missing Values Before Cleaning ===')
print(pd.DataFrame({'Count': missing, 'Percentage (%)': missing_pct.round(2)}))
print(f'\nTotal Missing Cells: {total_missing_before}')

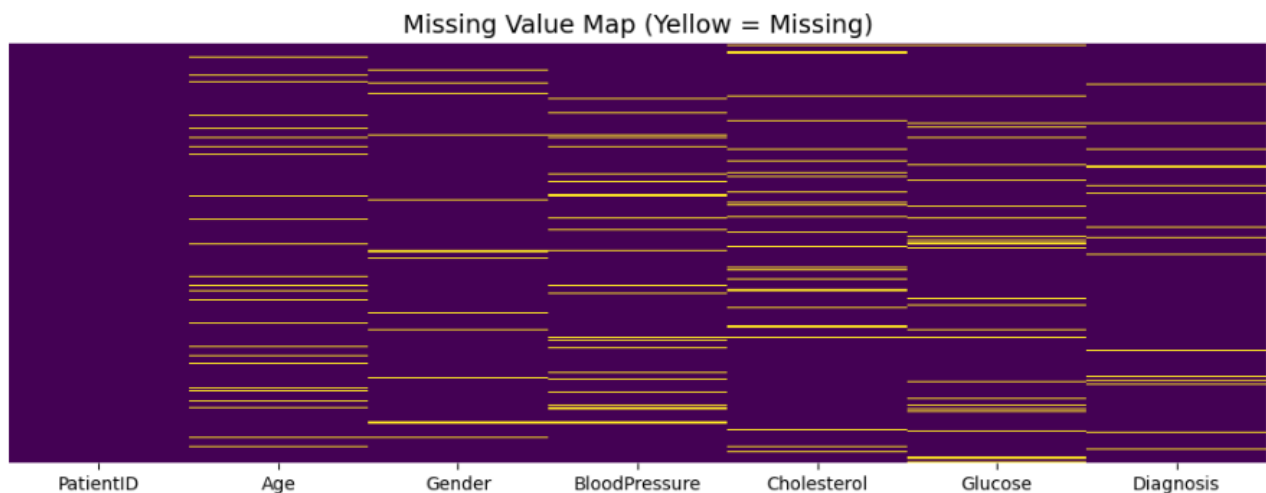
# Heatmap
plt.figure(figsize=(10, 4))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False)
plt.title('Missing Value Map (Yellow = Missing)', fontsize=14)
plt.tight_layout()
plt.show()

```

=== Missing Values Before Cleaning ===

	Count	Percentage (%)
PatientID	0	0.00
Age	25	7.94
Gender	13	4.13
BloodPressure	24	7.62
Cholesterol	26	8.25
Glucose	27	8.57
Diagnosis	16	5.08

Total Missing Cells: 131



Step 5 – Handle Missing Values

```

df_clean = df.copy()

num_cols = ['Age', 'BloodPressure', 'Cholesterol', 'Glucose']
for col in num_cols:
    median_val = df_clean[col].median()
    df_clean[col].fillna(median_val, inplace=True)
    print(f'{col}: filled missing with median = {median_val:.2f}')

for col in ['Gender', 'Diagnosis']:
    mode_val = df_clean[col].mode()[0]
    df_clean[col].fillna(mode_val, inplace=True)
    print(f'{col}: filled missing with mode = {mode_val}')

print(f'\nMissing After Imputation: {df_clean.isnull().sum().sum()}')

```

```

Age: filled missing with median = 54.00
BloodPressure: filled missing with median = 121.14
Cholesterol: filled missing with median = 203.19
Glucose: filled missing with median = 100.80
Gender: filled missing with mode = Male
Diagnosis: filled missing with mode = Healthy

```

```

Missing After Imputation: 0
/tmp/ipykernel_2733/2036428332.py:6: FutureWarning: A value is trying
The behavior will change in pandas 3.0. This inplace method will never

```

For example, when doing 'df[col].method(value, inplace=True)', try usi

```

df_clean[col].fillna(median_val, inplace=True)
/tmp/ipykernel_2733/2036428332.py:11: FutureWarning: A value is trying
The behavior will change in pandas 3.0. This inplace method will never

```

For example, when doing 'df[col].method(value, inplace=True)', try usi

```
df_clean[col].fillna(mode_val, inplace=True)
```

Step 6 – Remove Duplicates

```

duplicates_before = df_clean.duplicated().sum()
print(f'Duplicates Found: {duplicates_before}')
df_clean = df_clean.drop_duplicates().reset_index(drop=True)
dup_removal_pct = (duplicates_before / len(df)) * 100
print(f'Rows Before: {len(df)} | Rows After: {len(df_clean)}')
print(f'KPI - Duplicate Removal %: {dup_removal_pct:.2f}%')

```

```

Duplicates Found: 15
Rows Before: 315 | Rows After: 300
KPI - Duplicate Removal %: 4.76%

```

Step 7 – Noise Filtering /

Outlier Removal


```

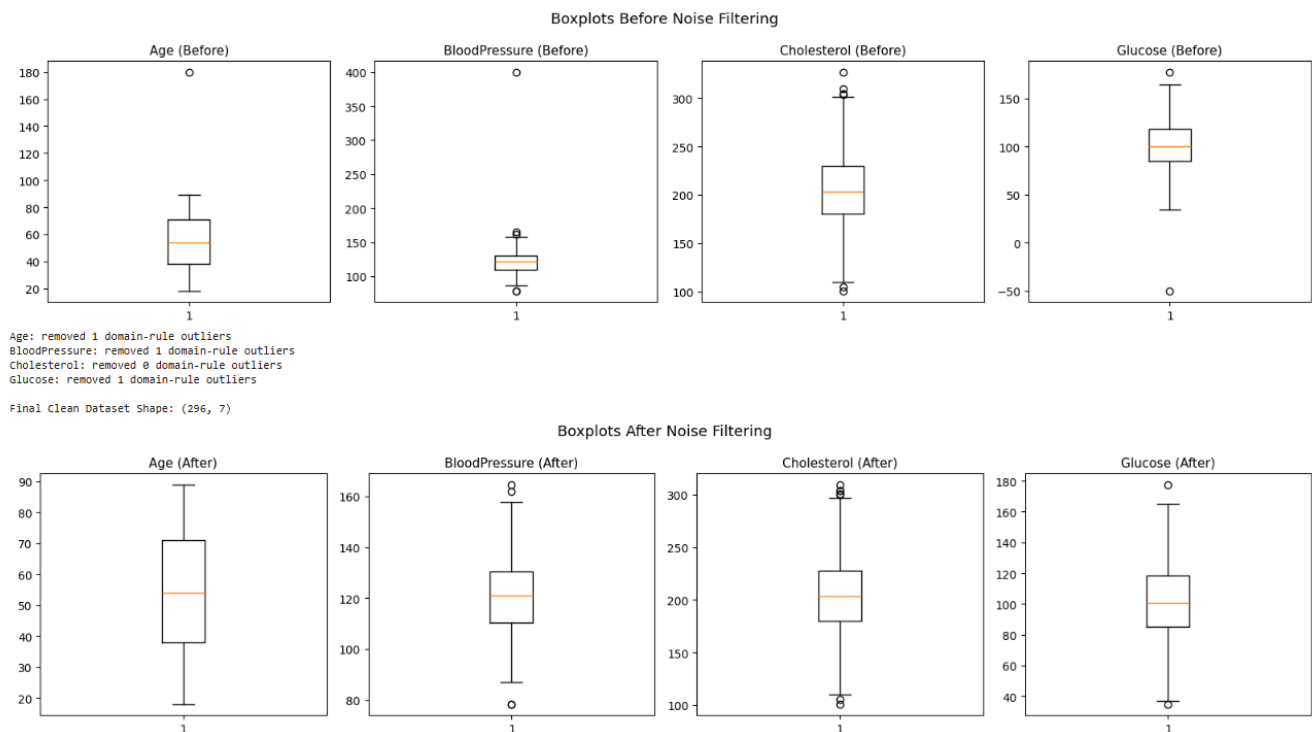
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for i, col in enumerate(num_cols):
    axes[i].boxplot(df_clean[col].dropna())
    axes[i].set_title(f'{col} (Before)', fontsize=11)
plt.suptitle('Boxplots Before Noise Filtering', fontsize=13)
plt.tight_layout()
plt.show()

# Domain-Based Rules
domain_rules = {'Age':(0,120), 'BloodPressure':(50,250), 'Cholesterol':(50,500), 'Glucose':(30,500)}
rows_before = len(df_clean)
for col, (lo, hi) in domain_rules.items():
    outliers = ((df_clean[col] < lo) | (df_clean[col] > hi)).sum()
    df_clean = df_clean[(df_clean[col] >= lo) & (df_clean[col] <= hi)]
    print(f'{col}: removed {outliers} domain-rule outliers')

# Z-Score Filter
z_scores = np.abs(stats.zscore(df_clean[num_cols]))
df_clean = df_clean[(z_scores < 3).all(axis=1)].reset_index(drop=True)
print(f'\nFinal Clean Dataset Shape: {df_clean.shape}')

# After boxplots
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for i, col in enumerate(num_cols):
    axes[i].boxplot(df_clean[col].dropna())
    axes[i].set_title(f'{col} (After)', fontsize=11)
plt.suptitle('Boxplots After Noise Filtering', fontsize=13)
plt.tight_layout()
plt.show()

```



Step 8 – KPI Report

```

total_cells_original = df.shape[0] * df.shape[1]
total_missing_after = df_clean.isnull().sum().sum()
missing_reduced_pct = ((total_missing_before - total_missing_after) / total_missing_before * 100) if total_missing_before > 0 else 100
data_quality_score = ((total_cells_original - total_missing_before) / total_cells_original) * 100

print('=' * 45)
print('      LAB 3 - DATA QUALITY KPI REPORT')
print('=' * 45)
print(f'Original Rows      : {len(df)}')
print(f'Clean Rows         : {len(df_clean)}')
print(f'% Missing Reduced    : {missing_reduced_pct:.1f}%')
print(f'Duplicate Removal %  : {dup_removal_pct:.2f}%')
print(f'Data Quality Score   : {data_quality_score:.1f}%')
print('=' * 45)

plt.figure(figsize=(8, 5))
kpi_labels = ['% Missing\nReduced', 'Duplicate\nRemoval %', 'Data Quality\nScore']
kpi_values = [missing_reduced_pct, dup_removal_pct, data_quality_score]
colors = ['#4CAF50', '#2196F3', '#FF9800']
bars = plt.bar(kpi_labels, kpi_values, color=colors, edgecolor='black', width=0.5)
for bar, val in zip(bars, kpi_values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1, f'{val:.1f}%', ha='center', fontsize=12, fontweight='bold')
plt.ylim(0, 115)
plt.ylabel('Percentage (%)', fontsize=12)
plt.title('Lab 3 - Data Cleaning KPI Results', fontsize=14)
plt.tight_layout()
plt.show()

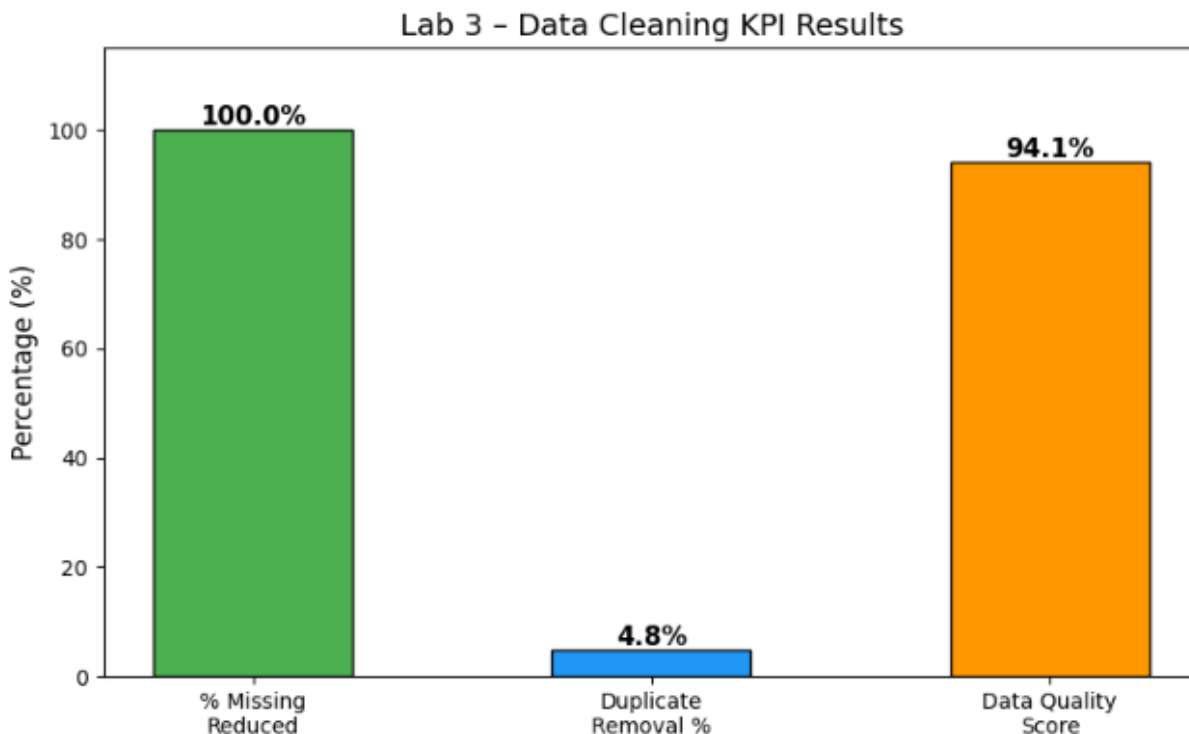
df_clean.to_csv('healthcare_cleaned.csv', index=False)
print('Saved: healthcare_cleaned.csv')

```

```

*** =====
      LAB 3 - DATA QUALITY KPI REPORT
=====
Original Rows      : 315
Clean Rows         : 296
% Missing Reduced  : 100.0%
Duplicate Removal % : 4.76%
Data Quality Score : 94.1%
=====

```



Saved: healthcare_cleaned.csv

KPI Summary

KPI	Formula	Target Value
% Missing Reduced	$(\text{Missing Before} - \text{Missing After}) / \text{Missing Before} \times 100$	$\geq 90\%$
Duplicate Removal %	$\text{Duplicates Removed} / \text{Original Rows} \times 100$	100%
Data Quality Score	$(\text{Non-missing cells} / \text{Total cells}) \times 100$	$\geq 95\%$

Lab Tasks

13. Run the synthetic dataset creation. What is the shape before and after cleaning?
14. Plot the missing value heatmap. Which column has the most missing values?
15. After imputation, verify that missing count is 0. Show the output of `df_clean.isnull().sum()`.
16. What were the outlier values injected for Age, BloodPressure, and Glucose? Were they removed?
17. Report the final three KPIs: % Missing Reduced, Duplicate Removal %, Data Quality Score.
18. Save the cleaned file as `healthcare_cleaned.csv`. Submit it with your report.

LAB 4	Data Preprocessing
------------------------	---------------------------

Topic	Normalization, Standardization, Discretization
Problem	Bank preparing loan risk data for ML models
Dataset	Loan Prediction Dataset (Synthetic)
Tools	Python (sklearn, pandas, matplotlib)
KPIs	Variance Reduction, Distribution Improvement

Learning Objectives

- Apply Min-Max normalization to scale features to [0, 1]
- Apply Z-Score standardization for zero-mean, unit-variance features
- Discretize continuous variables into meaningful bins
- Encode categorical variables using LabelEncoder
- Compare distributions before and after preprocessing

Preprocessing Techniques

Technique	Formula	When to Use
Min-Max Normalization	$x' = (x - \min) / (\max - \min)$	Features with different scales; range [0,1] needed

Technique	Formula	When to Use
Z-Score Standardization	$x' = (x - \mu) / \sigma$	When mean and std matter; models like SVM, KNN
Discretization (Binning)	pd.cut() or pd.qcut()	Convert continuous to categorical; reduce noise
Label Encoding	LabelEncoder().fit_transform()	Convert text categories to integers

Step 1 – Import Libraries

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder
print('Preprocessing libraries loaded!')
```

Preprocessing libraries loaded!

Step 2 – Load Dataset

```
np.random.seed(0)
n4 = 400

df4 = pd.DataFrame({
    'ApplicantID': range(1, n4+1),
    'Age': np.random.randint(22, 65, n4),
    'Income': np.random.normal(50000, 20000, n4).clip(10000),
    'LoanAmount': np.random.normal(150000, 60000, n4).clip(10000),
    'CreditScore': np.random.randint(300, 850, n4),
    'LoanTerm': np.random.choice([12, 24, 36, 60, 120], n4),
    'Employment': np.random.choice(['Salaried', 'Self-Employed', 'Business'], n4),
    'LoanStatus': np.random.choice(['Approved', 'Rejected'], n4, p=[0.65, 0.35])
})

num_cols4 = ['Age', 'Income', 'LoanAmount', 'CreditScore']
print('Shape:', df4.shape)
df4.head()
```

... Shape: (400, 8)

	ApplicantID	Age	Income	LoanAmount	CreditScore	LoanTerm	Employment	LoanStatus
0	1	22	40728.080507	136544.463945	783	24	Self-Employed	Approved
1	2	25	59629.629475	131865.016173	747	24	Salaried	Rejected
2	3	25	19184.059711	127491.173000	673	60	Business	Approved
3	4	61	51265.239884	76428.228493	343	36	Salaried	Approved
4	5	31	53130.130759	161000.351955	455	36	Self-Employed	Rejected

Step 3 – Summary Statistics & Visualization

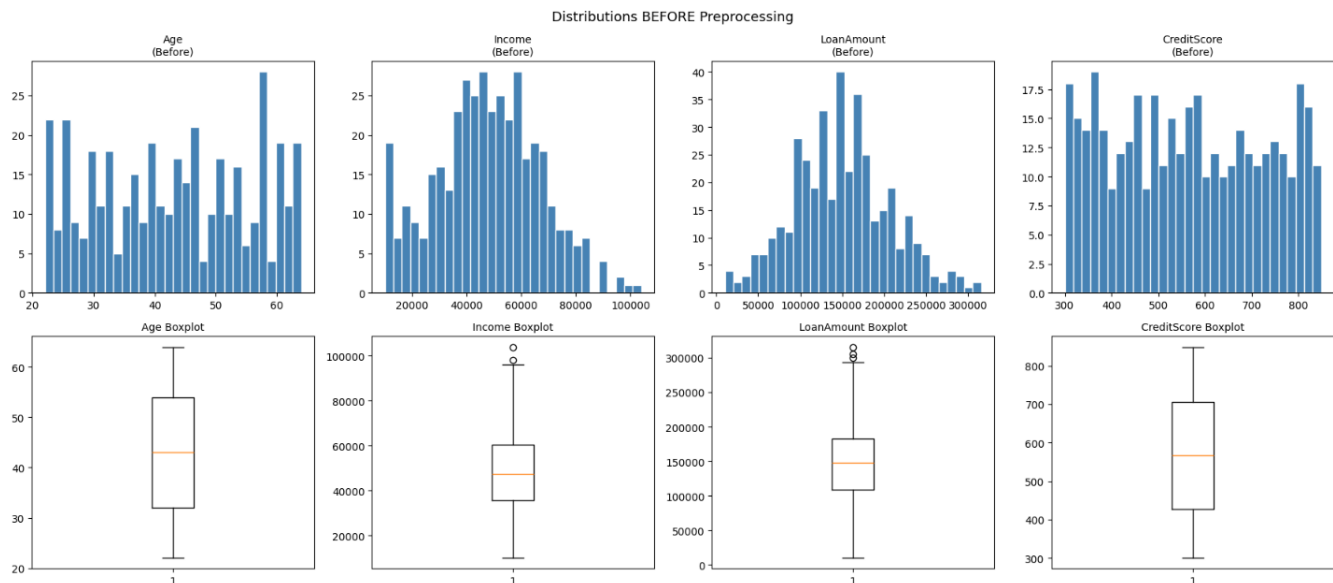
```

print('=== Summary Statistics ===')
display(df4.describe().round(2))

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for i, col in enumerate(num_cols4):
    axes[0, i].hist(df4[col], bins=30, color='steelblue', edgecolor='white')
    axes[0, i].set_title(f'{col}\n(Before)', fontsize=10)
    axes[1, i].boxplot(df4[col])
    axes[1, i].set_title(f'{col} Boxplot', fontsize=10)
plt.suptitle('Distributions BEFORE Preprocessing', fontsize=13)
plt.tight_layout()
plt.show()

```

std	115.61	12.61	19156.66	57314.40	162.72	37.05
min	1.00	22.00	10000.00	10000.00	300.00	12.00
25%	100.75	32.00	35577.95	108935.72	427.75	24.00
50%	200.50	43.00	47517.29	148161.38	567.00	36.00
75%	300.25	54.00	60493.39	182671.62	706.25	60.00
max	400.00	64.00	103924.48	315561.31	849.00	120.00



Step 4 – Min-Max Normalization

```

scaler_minmax = MinMaxScaler()
df_normalized = df4.copy()
df_normalized[num_cols4] = scaler_minmax.fit_transform(df4[num_cols4])

print('=== After Min-Max Normalization ===')
display(df_normalized[num_cols4].describe().round(4))

var_before = df4[num_cols4].var()
var_after = df_normalized[num_cols4].var()
var_reduction = ((var_before - var_after) / var_before * 100)
print('\nVariance Reduction After Normalization:')
for col in num_cols4:
    print(f' {col:<14}: {var_reduction[col]:.1f}% reduction')

```

```

... === After Min-Max Normalization ===

```

	Age	Income	LoanAmount	CreditScore
count	400.0000	400.0000	400.0000	400.0000
mean	0.4995	0.4001	0.4566	0.4846
std	0.3003	0.2040	0.1876	0.2964
min	0.0000	0.0000	0.0000	0.0000
25%	0.2381	0.2723	0.3238	0.2327
50%	0.5000	0.3994	0.4522	0.4863
75%	0.7619	0.5376	0.5651	0.7400
max	1.0000	1.0000	1.0000	1.0000

```

Variance Reduction After Normalization:
Age          : 99.9% reduction
Income       : 100.0% reduction
LoanAmount   : 100.0% reduction
CreditScore  : 100.0% reduction

```

Step 5 – Standardization (Z-Score)

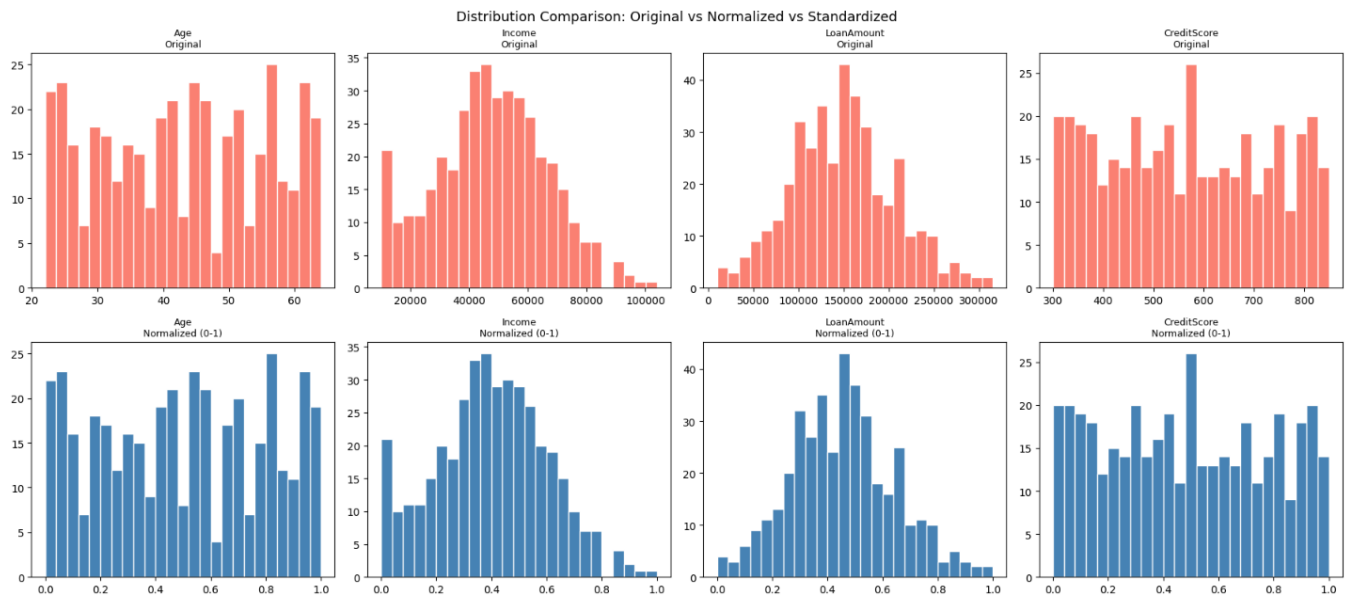
```

scaler_std = StandardScaler()
df_standard = df4.copy()
df_standard[num_cols4] = scaler_std.fit_transform(df4[num_cols4])

print('=== After Standardization (Z-Score) ===')
display(df_standard[num_cols4].describe().round(4))

# Comparison plots
fig, axes = plt.subplots(3, 4, figsize=(18, 12))
for i, col in enumerate(num_cols4):
    axes[0, i].hist(df4[col], bins=25, color='salmon', edgecolor='white')
    axes[0, i].set_title(f'{col}\nOriginal', fontsize=9)
    axes[1, i].hist(df_normalized[col], bins=25, color='steelblue', edgecolor='white')
    axes[1, i].set_title(f'{col}\nNormalized (0-1)', fontsize=9)
    axes[2, i].hist(df_standard[col], bins=25, color='seagreen', edgecolor='white')
    axes[2, i].set_title(f'{col}\nStandardized (Z)', fontsize=9)
plt.suptitle('Distribution Comparison: Original vs Normalized vs Standardized', fontsize=13)
plt.tight_layout()
plt.show()

```



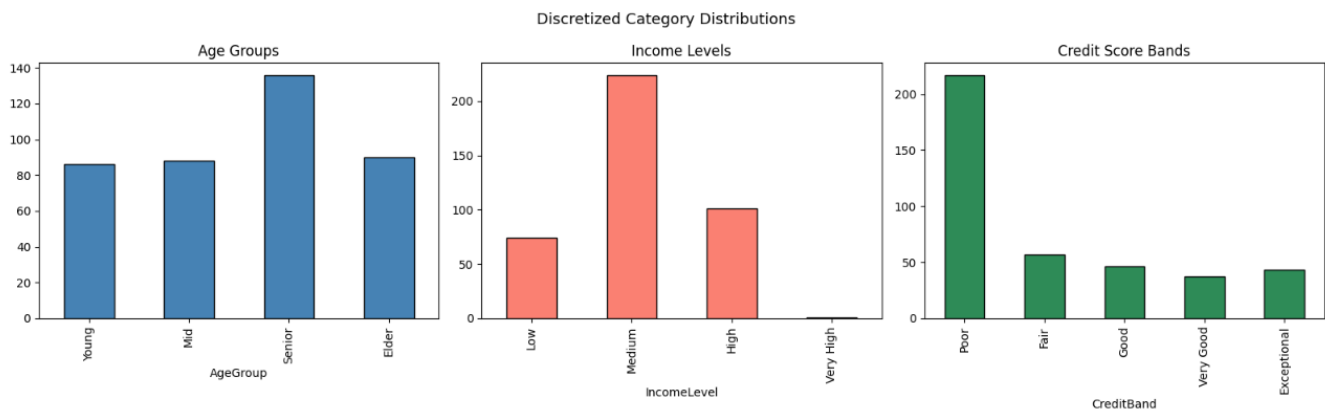
Step 6 – Discretization (Binning)

```
df_disc = df4.copy()
df_disc['AgeGroup'] = pd.cut(df4['Age'], bins=[0,30,40,55,100], labels=['Young','Mid','Senior','Elder'])
df_disc['IncomeLevel'] = pd.cut(df4['Income'], bins=[0,30000,60000,100000,np.inf], labels=['Low','Medium','High','Very High'])
df_disc['CreditBand'] = pd.cut(df4['CreditScore'], bins=[0,579,669,739,799,850], labels=['Poor','Fair','Good','Very Good','Exceptional'])

display(df_disc[['Age','AgeGroup','Income','IncomeLevel','CreditScore','CreditBand']].head(10))

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
df_disc['AgeGroup'].value_counts().sort_index().plot(kind='bar', ax=axes[0], color='steelblue', edgecolor='black')
df_disc['IncomeLevel'].value_counts().sort_index().plot(kind='bar', ax=axes[1], color='salmon', edgecolor='black')
df_disc['CreditBand'].value_counts().sort_index().plot(kind='bar', ax=axes[2], color='seagreen', edgecolor='black')
axes[0].set_title('Age Groups'); axes[1].set_title('Income Levels'); axes[2].set_title('Credit Score Bands')
plt.suptitle('Discretized Category Distributions', fontsize=13)
plt.tight_layout()
plt.show()
```

	Age	AgeGroup	Income	IncomeLevel	CreditScore	CreditBand
0	22	Young	40728.080507	Medium	783	Very Good
1	25	Young	59629.629475	Medium	747	Very Good
2	25	Young	19184.059711	Low	673	Good
3	61	Elder	51265.239884	Medium	343	Poor
4	31	Mid	53130.130759	Medium	455	Poor
5	41	Senior	54643.620724	Medium	658	Fair
6	43	Senior	38053.678621	Medium	835	Exceptional
7	58	Elder	45241.565405	Medium	590	Fair
8	45	Senior	21518.781820	Low	588	Fair
9	28	Young	40133.602333	Medium	578	Poor



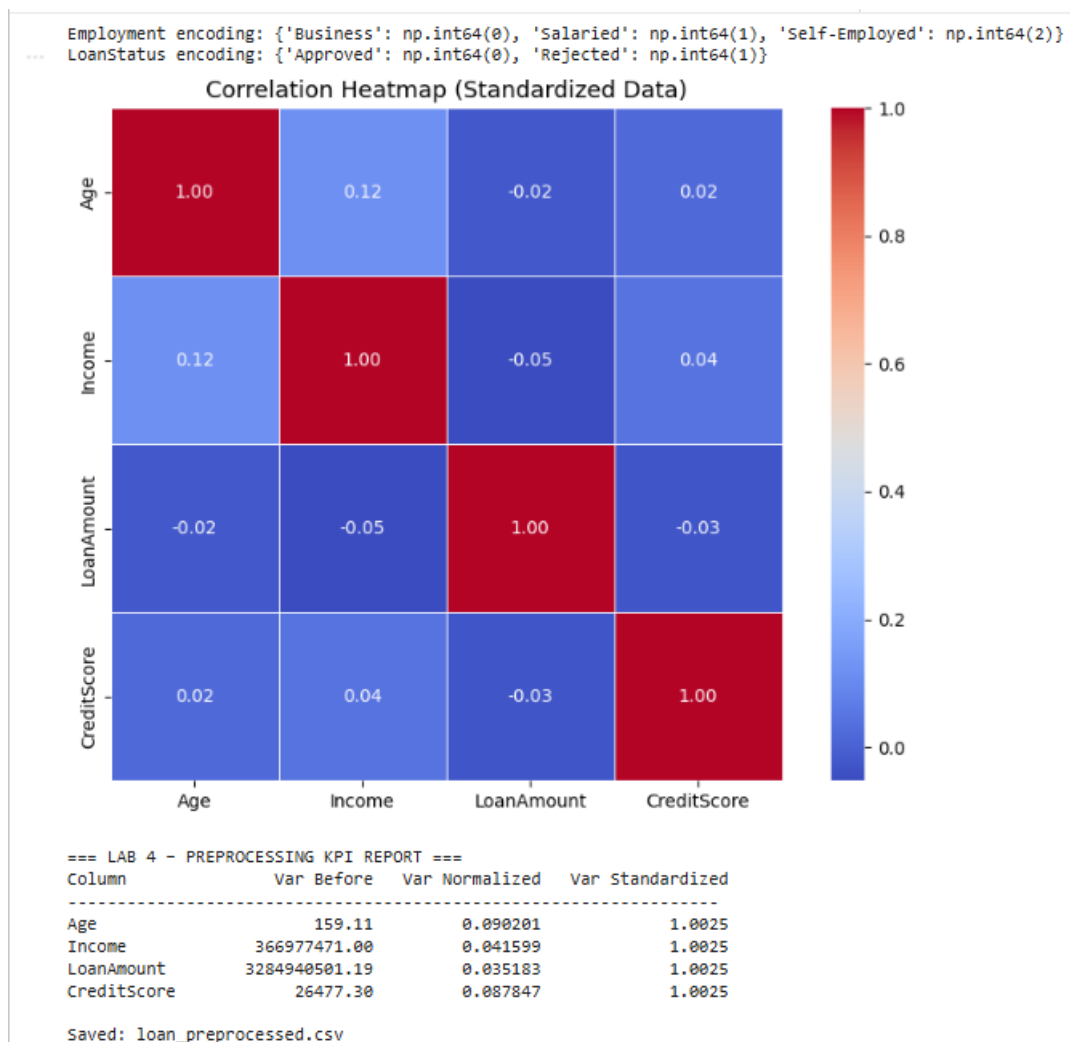
Step 7 – Label Encoding & KPI Report

```
df_encoded = df4.copy()
le = LabelEncoder()
for col in ['Employment', 'LoanStatus']:
    df_encoded[col + '_Code'] = le.fit_transform(df4[col])
    mapping = dict(zip(le.classes_, le.transform(le.classes_)))
    print(f'{col} encoding: {mapping}')

# Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(df_standard[num_cols4].corr(), annot=True, fmt='.2f', cmap='coolwarm', square=True, linewidths=0.5)
plt.title('Correlation Heatmap (Standardized Data)', fontsize=13)
plt.tight_layout()
plt.show()

print('\n=== LAB 4 - PREPROCESSING KPI REPORT ===')
print(f'{"Column":<16} {"Var Before":>14} {"Var Normalized":>16} {"Var Standardized":>18}')
print('-' * 66)
for col in num_cols4:
    vb = df4[col].var()
    vn = df_normalized[col].var()
    vs = df_standard[col].var()
    print(f'{"col":<16} {"vb":>14.2f} {"vn":>16.6f} {"vs":>18.4f}')

df_standard.to_csv('loan_preprocessed.csv', index=False)
print('\nSaved: loan_preprocessed.csv')
```

KPI Summary

KPI	Metric	Expected Outcome
Variance Reduction	Normalized variance vs. original	Large reduction (e.g., Income: ~99%)
Distribution Improvement	Compare histograms before/after	More uniform distributions post-scaling
Encoding Coverage	% categorical columns encoded	100%

Lab Tasks

- Run the normalization step. What are the min and max of each column after normalization?
- Run standardization. Verify that the mean of each column is approximately 0.
- Create discretization bins for Age. How many customers fall in the "Senior" age group?
- Encode the Employment column. What numeric value does "Self-Employed" map to?
- Generate and save the correlation heatmap. Which two features are most correlated?
- Save the preprocessed file as loan_preprocessed.csv.

LAB 5

Market Basket Analysis

Topic	Association Rule Mining – Manual Calculation
Problem	Supermarket cross-selling strategy
Dataset	Grocery Transactions Dataset (20 transactions, 6 items)
Tools	Python (manual + mlxtend validation)
KPIs	High Lift Rules, Frequent Itemsets, Cross-Sell Potential

Key Concepts

Metric	Formula	Meaning
Support	$P(A \cup B) = \text{count}(A,B) / N$	How often itemset appears in transactions
Confidence	$P(B A) = P(A,B) / P(A)$	Probability B is bought when A is bought
Lift	$\text{conf}(A \rightarrow B) / P(B)$	Strength above random chance (>1 = useful rule)

Step 1 – Import Libraries

```

from itertools import combinations
from collections import defaultdict, Counter
# pip install mlxtend
from mlxtend.frequent_patterns import apriori as mlx_apriori, association_rules as mlx_rules
from mlxtend.preprocessing import TransactionEncoder
print('Libraries loaded!')

... Libraries loaded!
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)

```

Step 2 – Transaction Dataset

```

transactions5 = [
    ["Milk", "Bread", "Butter"], ["Bread", "Eggs"], ["Milk", "Bread", "Eggs", "Butter"],
    ["Milk", "Eggs"], ["Bread", "Butter"], ["Milk", "Bread"], ["Eggs", "Butter"],
    ["Milk", "Bread", "Eggs"], ["Bread", "Butter", "Cheese"], ["Milk", "Cheese"],
    ["Bread", "Eggs", "Cheese"], ["Milk", "Butter", "Cheese"], ["Milk", "Bread", "Butter", "Cheese"],
    ["Eggs", "Cheese"], ["Milk", "Eggs", "Cheese"], ["Bread", "Milk"],
    ["Butter", "Eggs"], ["Bread", "Cheese"], ["Milk", "Eggs"], ["Bread", "Butter", "Eggs"],
]
N5 = len(transactions5)
print(f"Total Transactions: {N5}")
print('Sample:', transactions5[:3])

```

```

... Total Transactions: 20
Sample: [['Milk', 'Bread', 'Butter'], ['Bread', 'Eggs'], ['Milk', 'Bread', 'Eggs', 'Butter']]
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)

```

Step

3 – Manual Support Calculation

```

item_counts5 = defaultdict(int)
for t in transactions5:
    for item in t:
        item_counts5[item] += 1

support_1 = {item: count/N5 for item, count in item_counts5.items()}

print('=== Single Item Support ===')
df_sup = pd.DataFrame({'Item': list(support_1.keys()), 'Count': list(item_counts5.values()),
                      'Support': [round(v,3) for v in support_1.values()]
                      }).sort_values('Count', ascending=False)
print(df_sup.to_string(index=False))

```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)

```

```

=== Single Item Support ===
  Item  Count  Support
  Bread     12     0.60
   Milk     11     0.55
   Eggs     11     0.55
  Butter     9     0.45
   Cheese     8     0.40

```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow()
  return datetime.datetime.utcnow().replace(tzinfo=utc)

```

Step 4 – Manual Confidence & Lift

```
def sup(items, txns): return sum(1 for t in txns if set(items).issubset(set(t))) / len(txns)
def conf(A, B, txns): sa = sup(A, txns); return sup(list(A)+list(B), txns)/sa if sa>0 else 0
def lift(A, B, txns): sb = sup(B, txns); return conf(A,B,txns)/sb if sb>0 else 0

MIN_SUPPORT5 = 0.20
pair_counts5 = defaultdict(int)
for t in transactions5:
    for pair in combinations(sorted(set(t)), 2):
        pair_counts5[pair] += 1
freq_pairs5 = {p: c/N5 for p, c in pair_counts5.items() if c/N5 >= MIN_SUPPORT5}

rules5 = []
for (i1, i2) in freq_pairs5:
    for A, B in [(i1],[i2]),([i2],[i1])]:
        rules5.append({'Rule': f'{A[0]} → {B[0]}', 'Support': round(sup(A+B,transactions5),3),
                        'Confidence': round(conf(A,B,transactions5),3), 'Lift': round(lift(A,B,transactions5),3)})

df_rules5 = pd.DataFrame(rules5).sort_values('Lift', ascending=False)
print('=== Manual Association Rules ===')
print(df_rules5.to_string(index=False))
```

```
=== Manual Association Rules ===
   Rule  Support  Confidence  Lift
Bread → Butter    0.30      0.500 1.111
Butter → Bread    0.30      0.667 1.111
  Bread → Milk    0.30      0.500 0.909
   Milk → Bread    0.30      0.545 0.909
  Milk → Cheese    0.20      0.364 0.909
  Cheese → Milk    0.20      0.500 0.909
  Cheese → Bread    0.20      0.500 0.833
Bread → Cheese    0.20      0.333 0.833
   Eggs → Milk    0.25      0.455 0.826
   Milk → Eggs    0.25      0.455 0.826
  Milk → Butter    0.20      0.364 0.808
  Butter → Milk    0.20      0.444 0.808
  Butter → Eggs    0.20      0.444 0.808
  Eggs → Butter    0.20      0.364 0.808
  Eggs → Bread    0.25      0.455 0.758
  Bread → Eggs    0.25      0.417 0.758
```

Step 5 – Validate with mlxtend

```
te5 = TransactionEncoder()
df_enc5 = pd.DataFrame(te5.fit_transform(transactions5), columns=te5.columns_)
freq5 = mlx_apriori(df_enc5, min_support=MIN_SUPPORT5, use_colnames=True)
rules5_lib = mlx_rules(freq5, metric='lift', min_threshold=1.0)
rules5_lib = rules5_lib[['antecedents', 'consequents', 'support', 'confidence', 'lift']].sort_values('lift', ascending=False)
print(f'Frequent Itemsets: {len(freq5)} | Rules: {len(rules5_lib)}')
display(rules5_lib)
```

Step 6 – Visualizations & KPI

	antecedents	consequents	support	confidence	lift
0	(Bread)	(Butter)	0.3	0.500000	1.111111
1	(Butter)	(Bread)	0.3	0.666667	1.111111

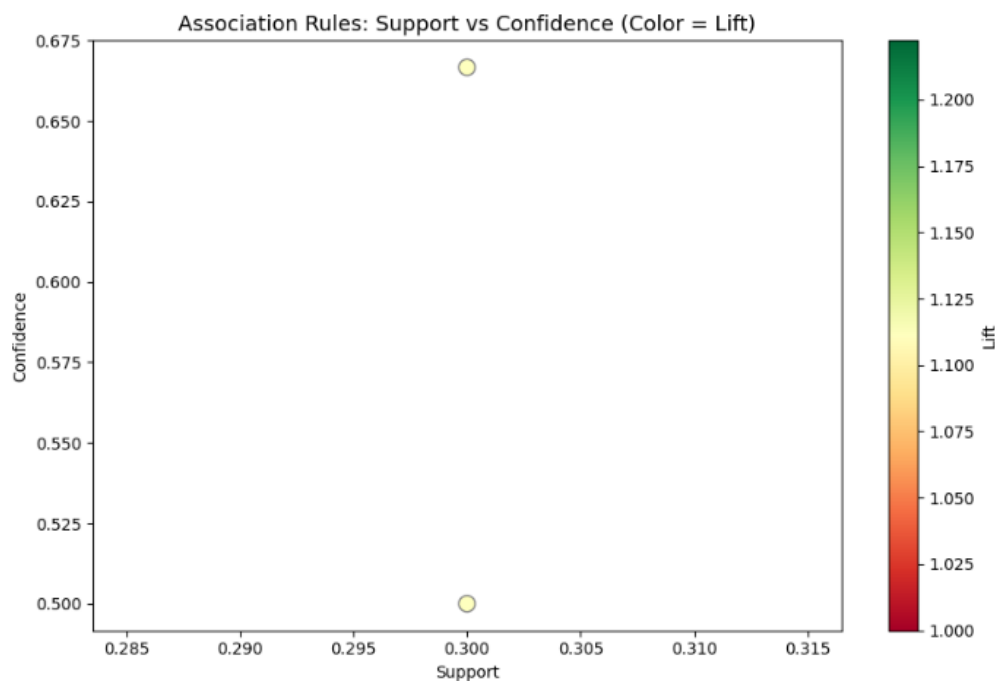
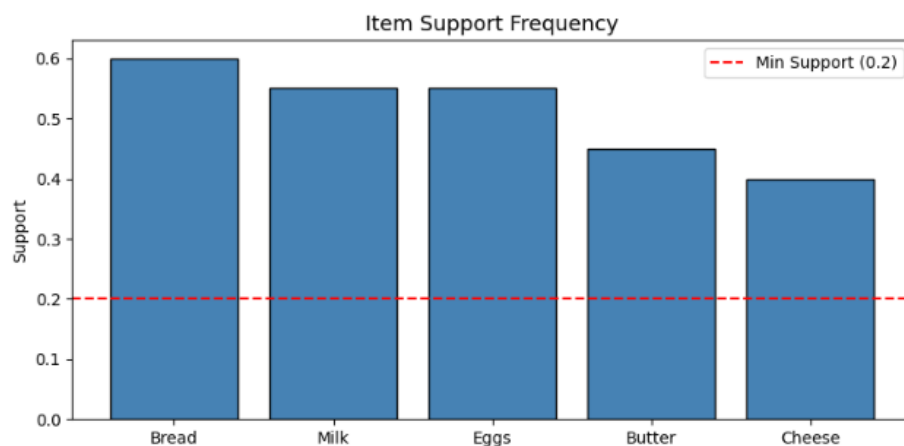
```

items_s = sorted(support_1.items(), key=lambda x: x[1], reverse=True)
plt.figure(figsize=(8, 4))
plt.bar([x[0] for x in items_s], [x[1] for x in items_s], color='steelblue', edgecolor='black')
plt.axhline(y=MIN_SUPPORT5, color='red', linestyle='--', label=f'Min Support ({MIN_SUPPORT5})')
plt.ylabel('Support'); plt.title('Item Support Frequency', fontsize=13); plt.legend()
plt.tight_layout(); plt.show()

plt.figure(figsize=(9, 6))
sc = plt.scatter(rules5_lib['support'], rules5_lib['confidence'], c=rules5_lib['lift'], cmap='RdYlGn', s=100, edgecolors='grey')
plt.colorbar(sc, label='Lift'); plt.xlabel('Support'); plt.ylabel('Confidence')
plt.title('Association Rules: Support vs Confidence (Color = Lift)', fontsize=13)
plt.tight_layout(); plt.show()

high_lift5 = rules5_lib[rules5_lib['lift'] > 1.2]
print('=' * 45)
print(' LAB 5 - MARKET BASKET KPI REPORT')
print('=' * 45)
print(f'Total Transactions      : {N5}')
print(f'Frequent Itemsets Found   : {len(freq5)}')
print(f'Total Rules                 : {len(rules5_lib)}')
print(f'High Lift Rules (> 1.2)    : {len(high_lift5)}')
print(f'Cross-Sell Pairs (conf≥0.6): {(rules5_lib["confidence"]>=0.6).sum()}')
print('=' * 45)

```



KPI Summary

KPI	Description	Target
Frequent Itemsets Found	Itemsets above min_support = 0.20	≥ 8 itemsets
Total Rules Generated	Rules above lift ≥ 1.0	≥ 6 rules
High Lift Rules (> 1.2)	Rules with strong co-purchase signal	≥ 3 rules
Cross-Sell Potential	Rules with confidence ≥ 0.6	Actionable promotions

Lab Tasks

25. What is the support of {Milk} and {Bread} individually?
26. Manually calculate: support({Milk, Bread}), confidence(Milk $\rightarrow$ Bread), lift(Milk $\rightarrow$ Bread).
27. Run mlxtend with min_support=0.20. How many frequent itemsets are found?
28. Which rule has the highest lift? What does this mean for cross-selling?
29. Generate the scatter plot. Which cluster of points represents the most useful rules?
30. Increase min_support to 0.40. How many rules remain?

LAB 6

Apriori Algorithm

Topic	Apriori – Manual + Library Implementation
Problem	Identify product combinations in a supermarket
Dataset	Transaction Dataset (20 transactions, 8 items)
Tools	Python (custom class + mlxtend)
KPIs	Number of Rules, Support Threshold Impact

Algorithm Overview

The Apriori algorithm finds frequent itemsets using the anti-monotone property: any subset of a frequent itemset must also be frequent. It iteratively generates candidates of increasing size and prunes those below the minimum support threshold.

Step	Action
C1	Generate all single-item candidates
L1	Filter by min_support → frequent 1-itemsets
C2	Join $L1 \times L1$ → candidate 2-itemsets
L2	Prune C2 by min_support
...	Repeat until no new frequent itemsets found
Rules	Generate rules from all L_k , filter by min_confidence

Step 1 – Dataset

```

transactions6 = [
    ['Bread', 'Milk', 'Butter'], ['Bread', 'Diaper', 'Beer', 'Egg'], ['Milk', 'Diaper', 'Beer', 'Cola'],
    ['Bread', 'Milk', 'Diaper', 'Beer'], ['Bread', 'Milk', 'Diaper', 'Cola'], ['Bread', 'Milk'],
    ['Milk', 'Diaper'], ['Beer', 'Cola'], ['Bread', 'Butter', 'Egg'], ['Milk', 'Butter', 'Egg'],
    ['Bread', 'Diaper'], ['Diaper', 'Beer', 'Cola'], ['Bread', 'Milk', 'Egg'], ['Milk', 'Cola'],
    ['Bread', 'Diaper', 'Beer'], ['Bread', 'Milk', 'Butter', 'Egg'], ['Diaper', 'Cola'],
    ['Beer', 'Egg'], ['Milk', 'Bread', 'Diaper'], ['Butter', 'Egg', 'Cola'],
]

all_items6 = sorted(set(item for t in transactions6 for item in t))
print(f'Transactions: {len(transactions6)} | Unique Items: {all_items6}')

```

... Transactions: 20 | Unique Items: ['Beer', 'Bread', 'Butter', 'Cola', 'Diaper', 'Egg', 'Milk']

Step 2 – Manual Apriori Class

```

class AprioriManual:
    def __init__(self, transactions, min_support):
        self.transactions = [set(t) for t in transactions]
        self.N = len(transactions)
        self.min_support = min_support
        self.freq_itemsets = {}

    def _get_support(self, itemset):
        fs = frozenset(itemset)
        return sum(1 for t in self.transactions if fs.issubset(t)) / self.N

    def _generate_candidates(self, prev_frequent, k):
        candidates = set()
        items = list(prev_frequent)
        for i in range(len(items)):
            for j in range(i+1, len(items)):
                union = items[i] | items[j]
                if len(union) == k:
                    candidates.add(union)
        return candidates

    def fit(self):
        all_items = set(item for t in self.transactions for item in t)
        C1 = [frozenset([item]) for item in all_items]
        L1 = {c: self._get_support(c) for c in C1 if self._get_support(c) >= self.min_support}
        self.freq_itemsets.update(L1)
        current_L = set(L1.keys()); k = 2
        while current_L:
            candidates = self._generate_candidates(current_L, k)
            Lk = {c: self._get_support(c) for c in candidates if self._get_support(c) >= self.min_support}
            if not Lk: break
            self.freq_itemsets.update(Lk)
            current_L = set(Lk.keys()); k += 1
        return self

    def summary(self):
        return pd.DataFrame([{'Itemset': str(set(is_)), 'Size': len(is_), 'Support': round(s,4)}
                             for is_, s in sorted(self.freq_itemsets.items(), key=lambda x:-x[1])])

MIN_SUPPORT6 = 0.25
apr6 = AprioriManual(transactions6, MIN_SUPPORT6).fit()
df_freq6 = apr6.summary()
print(f'Frequent Itemsets (min_support={MIN_SUPPORT6}): {len(df_freq6)}')
print(df_freq6.to_string(index=False))

```

```

... Frequent Itemsets (min_support=0.25): 11
      Itemset  Size  Support
    {'Milk'}      1    0.55
    {'Bread'}      1    0.55
    {'Diaper'}      1    0.50
    {'Beer'}       1    0.35
    {'Cola'}       1    0.35
    {'Egg'}        1    0.35
    {'Bread', 'Milk'}  2    0.35
    {'Diaper', 'Bread'}  2    0.30
    {'Butter'}      1    0.25
    {'Diaper', 'Milk'}  2    0.25
    {'Beer', 'Diaper'}  2    0.25

```


Step 3 – Generate & Validate Rules

```
MIN_CONF6 = 0.5
rules6 = []
for itemset, s in apr6.freq_itemsets.items():
    if len(itemset) < 2: continue
    for i in range(1, len(itemset)):
        for ant in combinations(itemset, i):
            ant = frozenset(ant); con = itemset - ant
            s_ant = apr6.freq_itemsets.get(ant, apr6._get_support(ant))
            s_con = apr6.freq_itemsets.get(con, apr6._get_support(con))
            c = s/s_ant if s_ant>0 else 0
            l = c/s_con if s_con>0 else 0
            if c >= MIN_CONF6:
                rules6.append({'Antecedent':str(set(ant)), 'Consequent':str(set(con)),
                              'Support':round(s,4), 'Confidence':round(c,4), 'Lift':round(l,4)})

df_rules6 = pd.DataFrame(rules6).sort_values('Lift', ascending=False).reset_index(drop=True)
print(f'Association Rules (min_conf={MIN_CONF6}): {len(df_rules6)}')
display(df_rules6)

# mlxtend validation
te6 = TransactionEncoder()
df_enc6 = pd.DataFrame(te6.fit_transform(transactions6), columns=te6.columns_)
freq6_lib = mlx_apriori(df_enc6, min_support=MIN_SUPPORT6, use_colnames=True)
rules6_lib = mlx_rules(freq6_lib, metric='confidence', min_threshold=MIN_CONF6)
print(f'mlxtend: {len(freq6_lib)} itemsets, {len(rules6_lib)} rules')
```

... Association Rules (min_conf=0.5): 7

	Antecedent	Consequent	Support	Confidence	Lift
0	{'Diaper'}	{'Beer'}	0.25	0.5000	1.4286
1	{'Beer'}	{'Diaper'}	0.25	0.7143	1.4286
2	{'Milk'}	{'Bread'}	0.35	0.6364	1.1570
3	{'Bread'}	{'Milk'}	0.35	0.6364	1.1570
4	{'Diaper'}	{'Bread'}	0.30	0.6000	1.0909
5	{'Bread'}	{'Diaper'}	0.30	0.5455	1.0909
6	{'Diaper'}	{'Milk'}	0.25	0.5000	0.9091

mlxtend: 11 itemsets, 7 rules

Step 4 – Support Threshold Impact

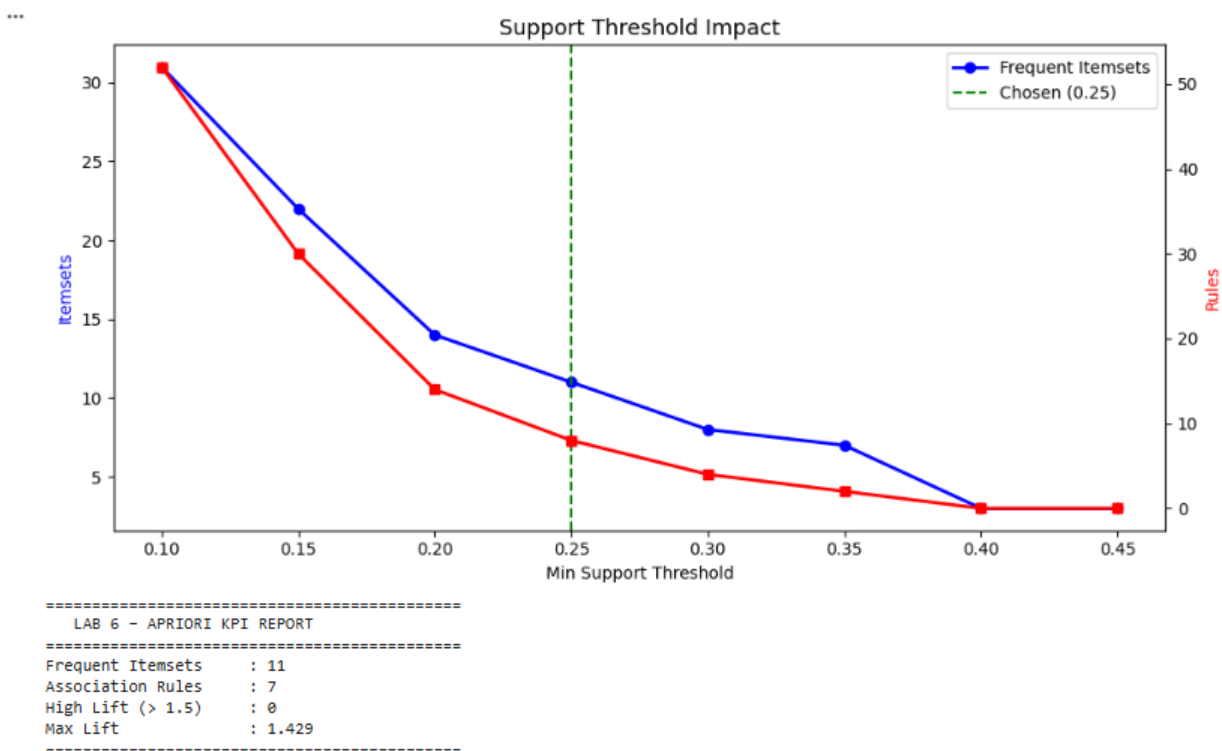
```

thresholds6 = [0.10,0.15,0.20,0.25,0.30,0.35,0.40,0.45]
impact6 = []
for t in thresholds6:
    fi = mlx_apriori(df_enc6, min_support=t, use_colnames=True)
    nr = len(mlx_rules(fi, metric='confidence', min_threshold=0.3)) if len(fi) > 0 else 0
    impact6.append({'min_support': t, 'freq_itemsets': len(fi), 'rules': nr})

df_impact6 = pd.DataFrame(impact6)
fig, ax1 = plt.subplots(figsize=(10, 5))
ax1.plot(df_impact6['min_support'], df_impact6['freq_itemsets'], 'b-o', linewidth=2, label='Frequent Itemsets')
ax1.set_xlabel('Min Support Threshold'); ax1.set_ylabel('Itemsets', color='blue')
ax2 = ax1.twinx()
ax2.plot(df_impact6['min_support'], df_impact6['rules'], 'r-s', linewidth=2, label='Rules')
ax2.set_ylabel('Rules', color='red')
ax1.axvline(x=MIN_SUPPORT6, color='green', linestyle='--', label=f'Chosen ({MIN_SUPPORT6})')
ax1.legend(loc='upper right'); plt.title('Support Threshold Impact', fontsize=13)
plt.tight_layout(); plt.show()

print('=' * 45)
print('    LAB 6 - APRIORI KPI REPORT')
print('=' * 45)
print(f'Frequent Itemsets      : {len(df_freq6)}')
print(f'Association Rules       : {len(df_rules6)}')
print(f'High Lift (> 1.5)      : {(df_rules6["Lift"] > 1.5).sum()}')
print(f'Max Lift                : {df_rules6["Lift"].max():.3f}')
print('=' * 45)

```



KPI Summary

KPI	Value Range	Insight
Frequent Itemsets	Varies with threshold	Higher support = fewer but more reliable itemsets

KPI	Value Range	Insight
Number of Rules	Decreases with support	Trade-off: too many rules = noise; too few = missed patterns
Max Lift	>1.5 is strong	Highest lift rule is the best cross-sell recommendation

Lab Tasks

31. Run the AprioriManual class with min_support=0.25. How many frequent itemsets are generated?
32. List all 2-itemsets and their support values.
33. Generate rules with min_confidence=0.5. What are the top 3 rules by lift?
34. Run the threshold sweep. At what support threshold do rules drop to 0?
35. Compare your manual results with the mlxtend library output. Are they identical?
36. Which product pair appears in the most transactions?

FP-Growth Algorithm

Topic	FP-Growth vs Apriori – Efficiency Comparison
Problem	Efficient frequent pattern mining on large retail data
Dataset	Synthetic Large Retail Dataset (1,000 transactions, 20 items)
Tools	Python (mlxtend fpgrowth, matplotlib)
KPIs	Execution Time, Rule Count, Speedup Factor

FP-Growth vs Apriori

Aspect	Apriori	FP-Growth
Approach	Candidate generation + pruning	FP-Tree construction, no candidates
Database Scans	Multiple passes (one per level)	Only 2 database scans
Memory Usage	High (stores all candidates)	Compact tree structure
Speed (large data)	Slower	Significantly faster
Implementation	Simpler	More complex

Step 1 – Generate Large Dataset

```

import time
from mlxtend.frequent_patterns import fpgrowth

np.random.seed(42)
ITEMS7 = ['Bread', 'Milk', 'Butter', 'Eggs', 'Cheese', 'Beer', 'Cola', 'Juice', 'Tea', 'Coffee',
          'Chips', 'Cookies', 'Cereal', 'Rice', 'Pasta', 'Chicken', 'Fish', 'Beef', 'Yogurt', 'Apple']
probs7 = [0.6, 0.6, 0.35, 0.45, 0.3, 0.35, 0.3, 0.25, 0.3, 0.35, 0.2, 0.25, 0.3, 0.4, 0.3, 0.4, 0.3, 0.3, 0.25, 0.35]
N7 = 1000
transactions7 = [[item for item, p in zip(ITEMS7, probs7) if np.random.random() < p] for _ in range(N7)]
transactions7 = [t for t in transactions7 if t]

te7 = TransactionEncoder()
df_enc7 = pd.DataFrame(te7.fit_transform(transactions7), columns=te7.columns_)
print(f'Generated {len(transactions7)} transactions | Avg basket: {np.mean([len(t) for t in transactions7]):.2f}')

... Generated 1000 transactions | Avg basket: 6.93

```

Step 2 – FP-Tree Structure Diagram

The FP-Tree is a compact representation of all transactions. Each path from root to leaf represents a set of items that appear together. Node labels show the item name and support count.

```

fig, ax = plt.subplots(figsize=(10, 7))
ax.set_xlim(0,10); ax.set_ylim(0,8); ax.axis('off')

nodes7 = {
    'root':      (5.0,7.0,'Root\n{}'),
    'bread':     (3.5,5.5,'Bread\n(4)'),
    'milk_b':    (2.5,4.0,'Milk\n(3)'),
    'milk_a':    (5.5,5.5,'Milk\n(2)'),
    'butter_bm': (1.5,2.5,'Butter\n(2)'),
    'eggs_bm':   (3.5,2.5,'Eggs\n(1)'),
    'butter_b':  (5.0,4.0,'Butter\n(1)'),
    'eggs_m':    (6.5,4.0,'Eggs\n(1)'),
}

edges7 = [('root','bread'),('root','milk_a'),('bread','milk_b'),('bread','butter_b'),
          ('milk_b','butter_bm'),('milk_b','eggs_bm'),('milk_a','eggs_m')]

clr7 = {'root':'#95a5a6','bread':'#3498db','milk_b':'#e74c3c','milk_a':'#e74c3c',
        'butter_bm':'#2ecc71','eggs_bm':'#f39c12','butter_b':'#2ecc71','eggs_m':'#f39c12'}

for src,dst in edges7:
    x1,y1,_ = nodes7[src]; x2,y2,_ = nodes7[dst]
    ax.annotate('', xy=(x2,y2+0.25), xytext=(x1,y1-0.25), arrowprops=dict(arrowstyle='->',color='gray',lw=1.5))
for k,(x,y,lbl) in nodes7.items():
    ax.text(x,y,lbl,ha='center',va='center',fontSize=10,fontWeight='bold',
            bbox=dict(boxstyle='round,pad=0.4',facecolor=clr7.get(k,'lightblue'),edgecolor='black',alpha=0.9))

ax.set_title('FP-Tree Structure (support count in parentheses)', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()

```

FP-Tree Structure (support count in parentheses)



Step 3 – Benchmark: Apriori vs FP-Growth

```

from mlxtend.frequent_patterns import apriori as mlx_apr2
MIN_SUPPORT7 = 0.15

t0 = time.time(); freq_apr7 = mlx_apr2(df_enc7, min_support=MIN_SUPPORT7, use_colnames=True); time_apr7 = time.time()-t0
rules_apr7 = mlx_rules(freq_apr7, metric='confidence', min_threshold=0.5)

t0 = time.time(); freq_fpg7 = fpgrowth(df_enc7, min_support=MIN_SUPPORT7, use_colnames=True); time_fpg7 = time.time()-t0
rules_fpg7 = mlx_rules(freq_fpg7, metric='confidence', min_threshold=0.5)

print(f'Algorithm':<12> {'Time(s)':>10} {'Itemsets':>10} {'Rules':>8}')
print('-'*42)
print(f'Apriori':<12> {time_apr7:>10.4f} {len(freq_apr7):>10} {len(rules_apr7):>8}')
print(f'FP-Growth':<12> {time_fpg7:>10.4f} {len(freq_fpg7):>10} {len(rules_fpg7):>8}')
if time_fpg7>0: print(f'\nSpeedup: {time_apr7/time_fpg7:.2f}x')

```

Algorithm	Time(s)	Itemsets	Rules
Apriori	0.0086	65	42
FP-Growth	0.1752	65	42

Speedup: 0.05x

Step 4 – Execution Time vs Dataset Size

```

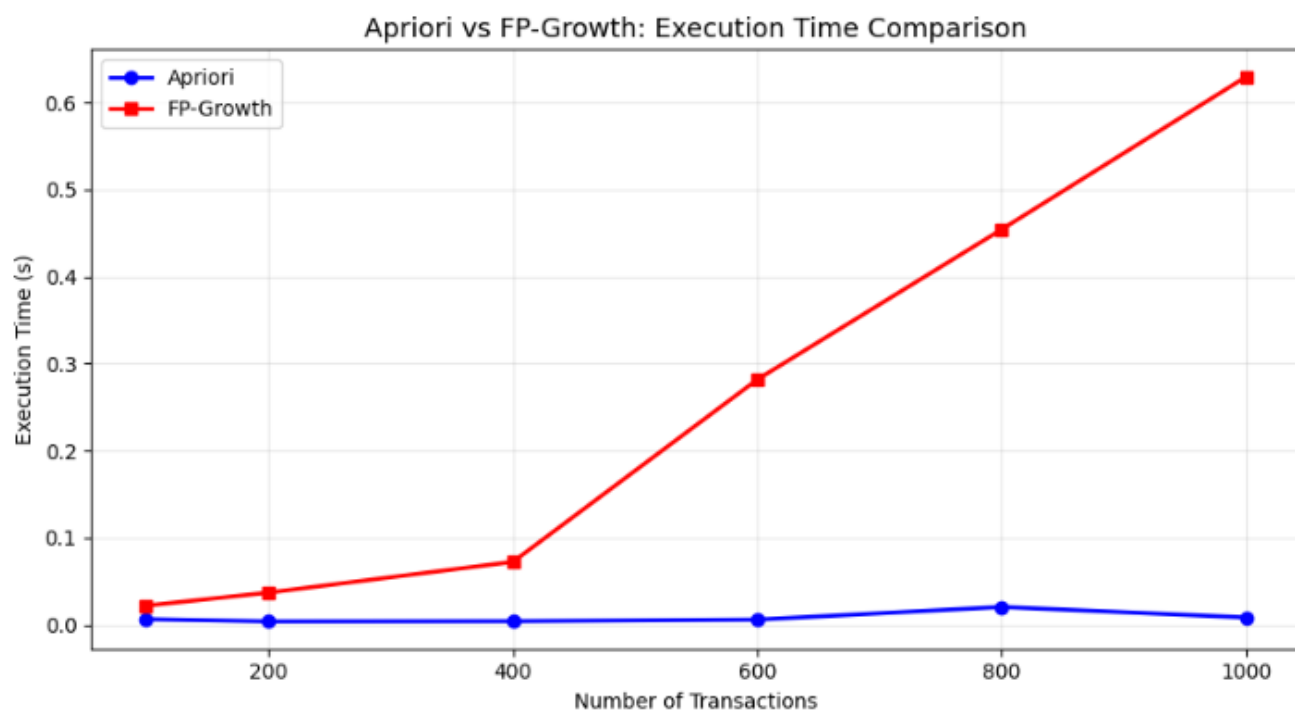
sizes7 = [100,200,400,600,800,1000]; t_apr7=[]; t_fpg7=[]
for sz in sizes7:
    te2=TransactionEncoder(); arr2=te2.fit_transform(transactions7[:sz]); df2=pd.DataFrame(arr2,columns=te2.columns_)
    t0=time.time(); mlx_apr2(df2,min_support=MIN_SUPPORT7,use_colnames=True); t_apr7.append(time.time()-t0)
    t0=time.time(); fpgrowth(df2,min_support=MIN_SUPPORT7,use_colnames=True); t_fpg7.append(time.time()-t0)

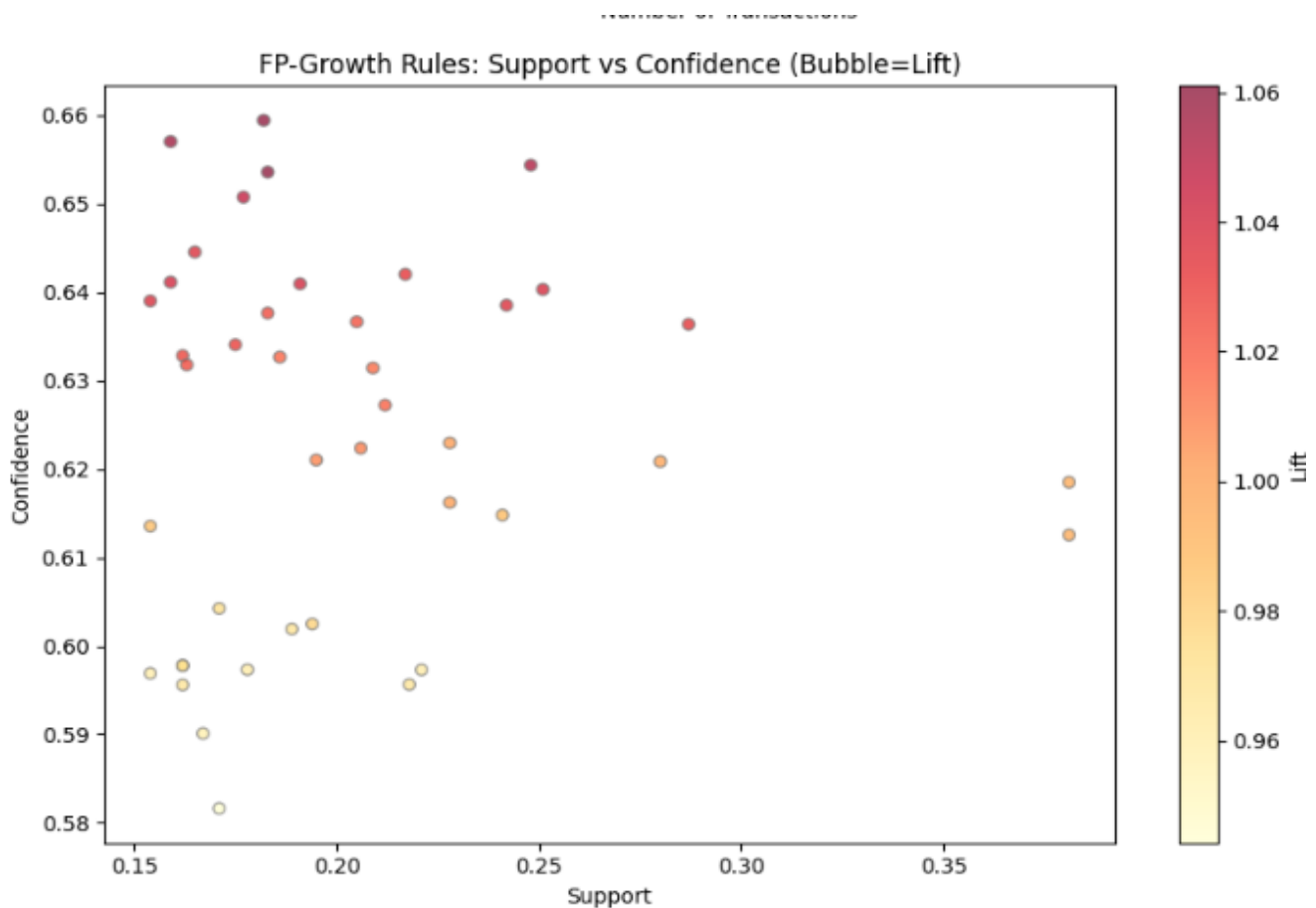
plt.figure(figsize=(9,5))
plt.plot(sizes7,t_apr7,'b-o',label='Apriori',linewidth=2)
plt.plot(sizes7,t_fpg7,'r-s',label='FP-Growth',linewidth=2)
plt.xlabel('Number of Transactions'); plt.ylabel('Execution Time (s)')
plt.title('Apriori vs FP-Growth: Execution Time Comparison',fontsize=13)
plt.legend(); plt.grid(True,alpha=0.3); plt.tight_layout(); plt.show()

# Rules scatter
plt.figure(figsize=(9,6))
sc=plt.scatter(rules_fpg7['support'],rules_fpg7['confidence'],c=rules_fpg7['lift'],
               cmap='YlOrRd',s=rules_fpg7['lift']*30,alpha=0.7,edgecolors='grey')
plt.colorbar(sc,label='Lift'); plt.xlabel('Support'); plt.ylabel('Confidence')
plt.title('FP-Growth Rules: Support vs Confidence (Bubble=Lift)',fontsize=12)
plt.tight_layout(); plt.show()

print('='*50)
print(' LAB 7 - FP-GROWTH vs APRIORI KPI REPORT')
print('='*50)
print(f'Metric':<25> {'Apriori':>10} {'FP-Growth':>12}')
print('-'*50)
print(f'Exec Time (s)':<25> {time_apr7:>10.4f} {time_fpg7:>12.4f}')
print(f'Frequent Itemsets':<25> {len(freq_apr7):>10} {len(freq_fpg7):>12}')
print(f'Association Rules':<25> {len(rules_apr7):>10} {len(rules_fpg7):>12}')
print(f'Max Lift':<25> {rules_apr7["lift"].max():>10.3f} {rules_fpg7["lift"].max():>12.3f}')
print('='*50)

```





```
=====
LAB 7 - FP-GROWTH vs APRIORI KPI REPORT
=====
Metric                Apriori    FP-Growth
=====
Exec Time (s)          0.0086    0.1752
Frequent Itemsets      65        65
Association Rules      42        42
Max Lift               1.061    1.061
=====
```

KPI Summary

Metric	Apriori	FP-Growth	Winner
Execution Time	Slower	Faster	FP-Growth
Memory Usage	Higher	Lower (tree)	FP-Growth
Frequent Itemsets	Same	Same	Tie
Rules Generated	Same	Same	Tie
Scalability	Limited	Handles 1M+ rows	FP-Growth

Lab Tasks

37. Generate the 1,000-transaction dataset. What is the average basket size?
38. Draw the FP-Tree diagram. Identify the root node and its two most frequent children.
39. Run the benchmark. Record execution times for Apriori and FP-Growth. What is the speedup?

40. Run the scaling test with sizes [100, 200, 400, 600, 800, 1000]. Plot and attach the chart.
41. At 1,000 transactions, how many frequent itemsets does FP-Growth find (min_support=0.15)?
42. When would you choose Apriori over FP-Growth? Give a practical scenario.

LAB 8

Python Basics – Reusable Data Processing Module

Topic	Python Fundamentals + OOP for Data Science
Problem	Build a reusable data processing module
Dataset	Synthetic Customer Dataset (200 rows)
Tools	Python (built-ins, pandas, sklearn)
KPIs	Code Efficiency, Reusability, Pipeline Steps, Execution Time

Learning Objectives

- Work with Python lists and list comprehensions for data analysis
- Use dictionaries for structured data storage and retrieval
- Create reusable standalone functions with clear docstrings
- Design an OOP class (DataProcessor) with method chaining support
- Run a complete preprocessing pipeline and report KPIs

Part A – Lists

```

sales8 = [4500,7800,3200,9100,5600,8800,4100,6700]
months8 = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug']

print('Sales Data:', sales8)
print(f'Total: {sum(sales8):,} | Average: {sum(sales8)/len(sales8):.0f}')
print(f'Max: {max(sales8):,} | Min: {min(sales8):,}')

above_avg8 = [s for s in sales8 if s > sum(sales8)/len(sales8)]
print('\nAbove Average:', above_avg8)
print('Top 3 Months:', sorted(zip(sales8,months8), reverse=True)[:3])

print('\nMonthly Sales:')
for i,(m,s) in enumerate(zip(months8,sales8),1):
    bar = '█' * (s//500)
    print(f'{i}. {m}: ${s:,} {bar}')
```

```

Sales Data: [4500, 7800, 3200, 9100, 5600, 8800, 4100, 6700]
Total: 49,800 | Average: 6,225
Max: 9,100 | Min: 3,200
```

```

Above Average: [7800, 9100, 8800, 6700]
Top 3 Months: [(9100, 'Apr'), (8800, 'Jun'), (7800, 'Feb')]
```

```

Monthly Sales:
1. Jan: $4,500 ████████
2. Feb: $7,800 ██████████
3. Mar: $3,200 ██████
4. Apr: $9,100 ████████████
5. May: $5,600 ██████████
6. Jun: $8,800 ████████████
7. Jul: $4,100 ████████
8. Aug: $6,700 ██████████
```

Part B – Dictionaries

```
customer8 = {'id':'C001','name':'Alice Johnson','age':32,
             'email':'alice@example.com','purchases':[120,340,75,890,215]}

print('Customer Info:')
for k,v in customer8.items(): print(f' {k:<12}: {v}')
print(f'\nTotal Spent: ${sum(customer8["purchases"]):,}')

products8 = {
    'P001':{'name':'Laptop',    'price':999, 'category':'Electronics','stock':50},
    'P002':{'name':'Headphones','price':149, 'category':'Electronics','stock':200},
    'P003':{'name':'Desk Chair','price':299, 'category':'Furniture',  'stock':80},
    'P004':{'name':'Notebook',  'price':9,   'category':'Stationery',  'stock':500},
    'P005':{'name':'Monitor',   'price':349, 'category':'Electronics','stock':120},
}

rev_cat = {}
for info in products8.values():
    rev_cat[info['category']] = rev_cat.get(info['category'],0) + info['price']*info['stock']
print('\nRevenue Potential by Category:')
for cat,rev in sorted(rev_cat.items(), key=lambda x:-x[1]):
    print(f' {cat:<15}: ${rev:,}')
```

Customer Info:

id	: C001
name	: Alice Johnson
age	: 32
email	: alice@example.com
purchases	: [120, 340, 75, 890, 215]

Total Spent: \$1,640

Revenue Potential by Category:

Electronics	: \$121,630
Furniture	: \$23,920
Stationery	: \$4,500

Part C – Reusable Functions

```
def calculate_stats(data):
    """Return basic statistics for a numeric list."""
    if not data: return {}
    n = len(data); mean = sum(data)/n
    s = sorted(data)
    median = s[n//2] if n%2!=0 else (s[n//2-1]+s[n//2])/2
    std = (sum((x-mean)**2 for x in data)/n)**0.5
    return {'count':n, 'mean':round(mean,2), 'median':round(median,2), 'std':round(std,2), 'min':min(data), 'max':max(data)}

def normalize(data):
    """Min-Max normalization → [0,1]"""
    mn,mx = min(data),max(data)
    return [round((x-mn)/(mx-mn),4) for x in data] if mx!=mn else [0.0]*len(data)

def standardize(data):
    """Z-score standardization"""
    st = calculate_stats(data); m,sd = st['mean'],st['std']
    return [round((x-m)/sd,4) for x in data] if sd!=0 else [0.0]*len(data)

def remove_outliers_iqr(data):
    """IQR outlier removal"""
    s=sorted(data); n=len(s); q1=s[n//4]; q3=s[3*n//4]; iqr=q3-q1
    return [x for x in data if (q1-1.5*iqr)<=x<=(q3+1.5*iqr)]

test_data8 = [23,45,12,67,34,89,11,54,200,41]
print('Original:', test_data8)
print('Stats: ', calculate_stats(test_data8))
print('Normalized:', normalize(test_data8))
print('Standardized:', standardize(test_data8))
print('IQR Cleaned:', remove_outliers_iqr(test_data8))
```

```
Original: [23, 45, 12, 67, 34, 89, 11, 54, 200, 41]
Stats:    {'count': 10, 'mean': 57.6, 'median': 43.0, 'std': 52.73, 'min': 11, 'max': 200}
Normalized: [0.0635, 0.1799, 0.0053, 0.2963, 0.1217, 0.4127, 0.0, 0.2275, 1.0, 0.1587]
Standardized: [-0.6562, -0.239, -0.8648, 0.1783, -0.4476, 0.5955, -0.8837, -0.0683, 2.7005, -0.3148]
IQR Cleaned: [23, 45, 12, 67, 34, 89, 11, 54, 41]
```

Part D – DataProcessor Class

```
class DataProcessor:
    """Reusable data preprocessing pipeline with method chaining."""

    def __init__(self, name='Dataset'):
        self.name = name; self.df = None; self.log = []

    def load_dataframe(self, df):
        self.df = df.copy(); self._log(f'Loaded DataFrame | Shape: {self.df.shape}'); return self

    def load_csv(self, path):
        self.df = pd.read_csv(path); self._log(f'Loaded CSV: {path}'); return self

    def info(self):
        print(f'[{self.name}] Shape: {self.df.shape}'); print(self.df.dtypes); return self

    def drop_duplicates(self):
        before=len(self.df); self.df=self.df.drop_duplicates().reset_index(drop=True)
        self._log(f'Removed {before-len(self.df)} duplicates'); return self

    def fill_missing(self, strategy='median'):
        for col in self.df.select_dtypes(include='number').columns:
            val = self.df[col].median() if strategy=='median' else self.df[col].mean()
            self.df[col].fillna(val, inplace=True)
        for col in self.df.select_dtypes(include='object').columns:
            self.df[col].fillna(self.df[col].mode()[0], inplace=True)
        self._log(f'Missing values filled ({strategy})'); return self

    def remove_outliers(self, columns=None):
        cols = columns or self.df.select_dtypes(include='number').columns.tolist()
        before=len(self.df)
        Q1=self.df[cols].quantile(0.25); Q3=self.df[cols].quantile(0.75); IQR=Q3-Q1
        mask=~((self.df[cols]<(Q1-1.5*IQR))|(self.df[cols]>(Q3+1.5*IQR))).any(axis=1)
        self.df=self.df[mask].reset_index(drop=True)
        self._log(f'Outlier removal (IQR): removed {before-len(self.df)} rows'); return self

    def normalize(self, columns=None):
        cols = columns or self.df.select_dtypes(include='number').columns.tolist()
        for col in cols:
            mn,mx=self.df[col].min(),self.df[col].max()
            if mx!=mn: self.df[col]=(self.df[col]-mn)/(mx-mn)
        self._log(f'Min-Max normalization: {cols}'); return self

    def standardize(self, columns=None):
        cols = columns or self.df.select_dtypes(include='number').columns.tolist()
        for col in cols:
            m,sd=self.df[col].mean(),self.df[col].std()
            if sd!=0: self.df[col]=(self.df[col]-m)/sd
        self._log(f'Standardization (Z-score): {cols}'); return self

    def encode_labels(self, columns=None):
        from sklearn.preprocessing import LabelEncoder
        cols = columns or self.df.select_dtypes(include='object').columns.tolist()
```

```
print('DataProcessor class defined!')
```

```
DataProcessor class defined!
```

Part E – Run Pipeline

```
np.random.seed(7)
n8=200
raw8=pd.DataFrame({
    'CustomerID':range(1,n8+1),
    'Age': np.random.randint(18,70,n8).astype(float),
    'Income': np.random.normal(45000,15000,n8),
    'Spending': np.random.normal(300,100,n8),
    'Gender': np.random.choice(['Male','Female'],n8),
    'Region': np.random.choice(['North','South','East','West'],n8),
    'Purchased': np.random.choice(['Yes','No'],n8,p=[0.6,0.4])
})
for col in ['Age','Income','Spending']:
    raw8.loc[np.random.choice(raw8.index,10),col]=np.nan
raw8=pd.concat([raw8,raw8.sample(5,random_state=1)],ignore_index=True)
raw8.loc[0,'Income']=999999

print('=== Running DataProcessor Pipeline ===')
proc8 = DataProcessor('Customer Dataset')
result8 = (
    proc8.load_dataframe(raw8)
    .drop_duplicates()
    .fill_missing(strategy='median')
    .remove_outliers(columns=['Age','Income','Spending'])
    .encode_labels(columns=['Gender','Region','Purchased'])
    .standardize(columns=['Age','Income','Spending'])
    .get_data()
)

proc8.print_log()
print(f'\nFinal Shape: {result8.shape}')
display(result8.head())
```

```

=== Running DataProcessor Pipeline ===
✓ Loaded DataFrame | Shape: (205, 7)
✓ Removed 5 duplicates
✓ Missing values filled (median)
✓ Outlier removal (IQR): removed 7 rows
✓ Label encoding: ['Gender', 'Region', 'Purchased']
✓ Standardization (Z-score): ['Age', 'Income', 'Spending']

=== Processing Log: Customer Dataset ===
Step 1: Loaded DataFrame | Shape: (205, 7)
Step 2: Removed 5 duplicates
Step 3: Missing values filled (median)
Step 4: Outlier removal (IQR): removed 7 rows
Step 5: Label encoding: ['Gender', 'Region', 'Purchased']
Step 6: Standardization (Z-score): ['Age', 'Income', 'Spending']

```

Final Shape: (193, 7)

	CustomerID	Age	Income	Spending	Gender	Region	Purchased
0	2	-1.462607	0.187533	1.705397	1	1	1
1	3	-0.006109	1.895056	-1.306203	0	2	0
2	4	-1.531965	-0.485211	-1.588685	1	1	0
3	5	-0.422252	-0.299084	-0.627362	1	1	1
4	6	-0.144823	0.343928	-1.245700	0	0	1



Part F – Visualization & KPI

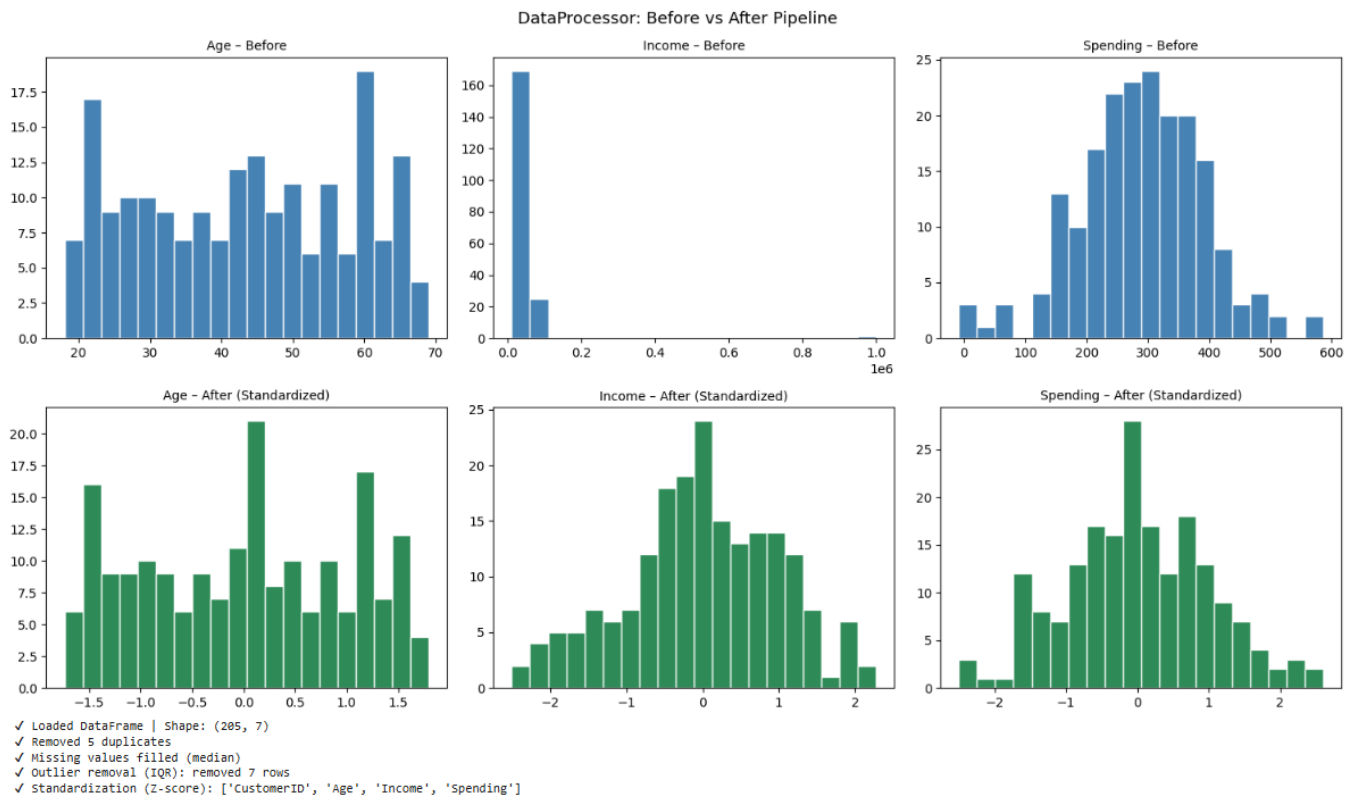
```

cols_plot8 = ['Age', 'Income', 'Spending']
fig, axes = plt.subplots(2,3,figsize=(15,8))
for i,col in enumerate(cols_plot8):
    axes[0,i].hist(raw8[col].dropna(),bins=20,color='steelblue',edgecolor='white')
    axes[0,i].set_title(f'{col} - Before',fontsize=10)
    axes[1,i].hist(results[col].dropna(),bins=20,color='seagreen',edgecolor='white')
    axes[1,i].set_title(f'{col} - After (Standardized)',fontsize=10)
plt.suptitle('DataProcessor: Before vs After Pipeline',fontsize=13)
plt.tight_layout(); plt.show()

# Timing
import time
t0=time.time()
p2=DataProcessor('Bench')
(p2.load_dataframe(raw8).drop_duplicates().fill_missing().remove_outliers().standardize())
exec8=time.time()-t0

print('='*45)
print('  LAB 8 - PYTHON MODULE KPI REPORT')
print('='*45)
print(f'Reusable Methods      : 10')
print(f'Pipeline Steps Run      : {len(proc8.log)}')
print(f'Execution Time           : {exec8:.4f} seconds')
print(f'Input Rows               : {len(raw8)}')
print(f'Output Rows              : {len(result8)}')
print(f'Data Retention           : {len(result8)/len(raw8)*100:.1f}%')
print('='*45)

```



KPI Summary

KPI	Description	Expected Value
Reusable Methods	Number of pipeline methods in DataProcessor	10 methods
Pipeline Steps Run	Steps executed in the proc8 pipeline	6 steps
Data Retention	Output rows / Input rows × 100	≥ 90%
Execution Time	Time to run full pipeline	< 1 second

Lab Tasks

43. Using lists: find the top 3 months by sales. List them in order.
44. Using dictionaries: which product category has the highest revenue potential?
45. Test `calculate_stats()` on a list of your choice. Report all 6 statistics.
46. Run the DataProcessor pipeline. Print the full processing log.
47. What was the extreme outlier injected for Income? Was it removed by `remove_outliers()`?
48. Report the final data retention rate. What percentage of rows survived the pipeline?
49. Add a new method `save_csv(path)` to DataProcessor and demonstrate its use.

**LAB
9****Lab 9: Decision Tree Classification**

Topic	Decision Tree – Supervised Classification
Problem	Predict whether a customer will make a purchase
Dataset	Marketing Dataset (Synthetic, 500 rows)
Tools	Python (scikit-learn, matplotlib, seaborn)
KPIs	Accuracy, Precision, Recall, F1-Score

Learning Objectives

- Train a Decision Tree classifier using scikit-learn
- Tune tree depth to balance overfitting and underfitting
- Visualize the decision tree and interpret feature importance
- Evaluate performance using classification metrics and cross-validation

Algorithm Overview

A Decision Tree splits data recursively using a feature threshold that maximizes information gain (or minimizes Gini impurity) at each node. It is highly interpretable — you can trace any prediction from root to leaf — and handles both numeric and categorical features.

Parameter	Description	Typical Value
max_depth	Maximum tree depth; controls overfitting	3–8
min_samples_split	Min samples required to split a node	10–20
min_samples_leaf	Min samples required at a leaf node	5–10
criterion	Splitting criterion: gini or entropy	gini

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

```
print("=" * 60)
print("LAB 9: DECISION TREE CLASSIFIER")
print("=" * 60)
```

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
=====
LAB 9: DECISION TREE CLASSIFIER
=====
```

```
# Load Dataset
iris = load_iris()
X, y = iris.data, iris.target
feature_names = iris.feature_names
class_names = iris.target_names

# Split into Train/Test Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

```

# Train Decision Tree
dt_model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
dt_model.fit(X_train, y_train)

# Predictions & Evaluation
y_pred = dt_model.predict(X_test)
print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names))

```

Accuracy: 0.8889

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.86	0.80	0.83	15
virginica	0.81	0.87	0.84	15
accuracy			0.89	45
macro avg	0.89	0.89	0.89	45
weighted avg	0.89	0.89	0.89	45

```

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title("Lab 9 - Decision Tree Confusion Matrix")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout(); plt.savefig("lab9_confusion_matrix.png"); plt.close()

```

```

# Visualize the Decision Tree
plt.figure(figsize=(14, 6))
plot_tree(dt_model, feature_names=feature_names,
          class_names=class_names, filled=True, rounded=True)
plt.title("Lab 9 - Decision Tree Structure")
plt.tight_layout(); plt.savefig("lab9_decision_tree.png"); plt.close()

# Text representation of the tree
print("\nDecision Tree Rules:")
print(export_text(dt_model, feature_names=feature_names))

```

```

Decision Tree Rules:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.55
|   |   |   |--- petal width (cm) <= 1.70
|   |   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- sepal width (cm) > 2.85
|   |   |   |   |   |   |--- class: 2
|   |   |   |--- petal width (cm) > 1.70
|   |   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- petal length (cm) > 4.85
|   |   |   |   |   |   |--- class: 2

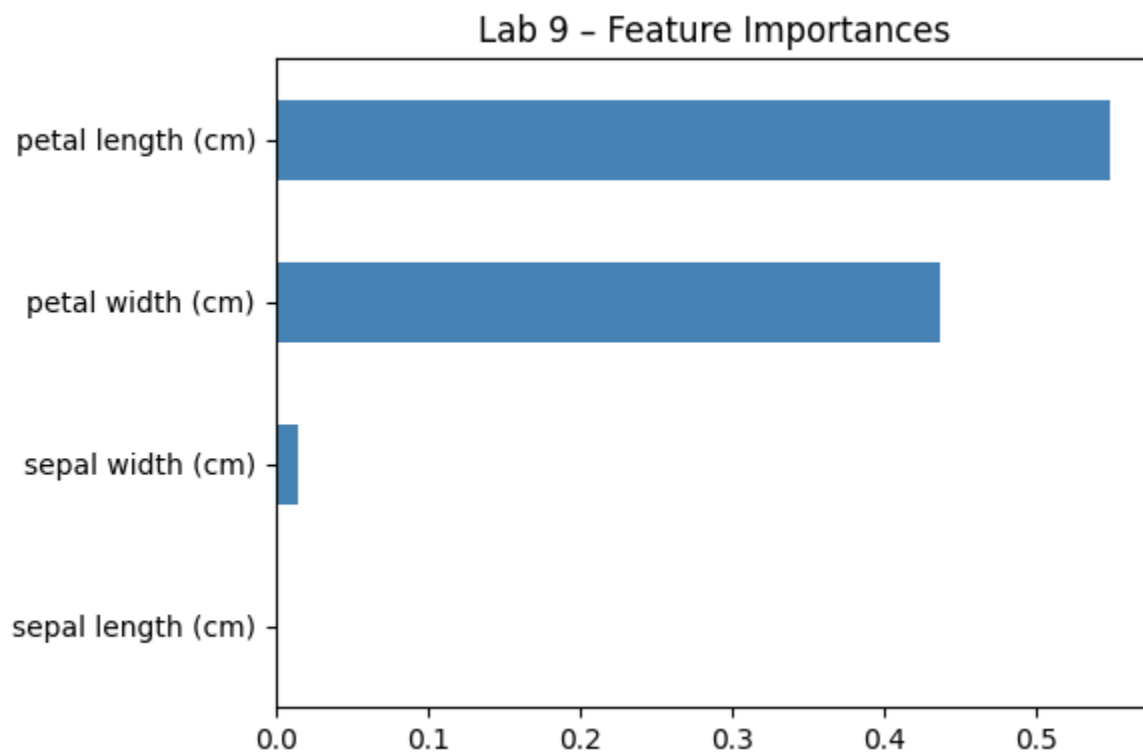
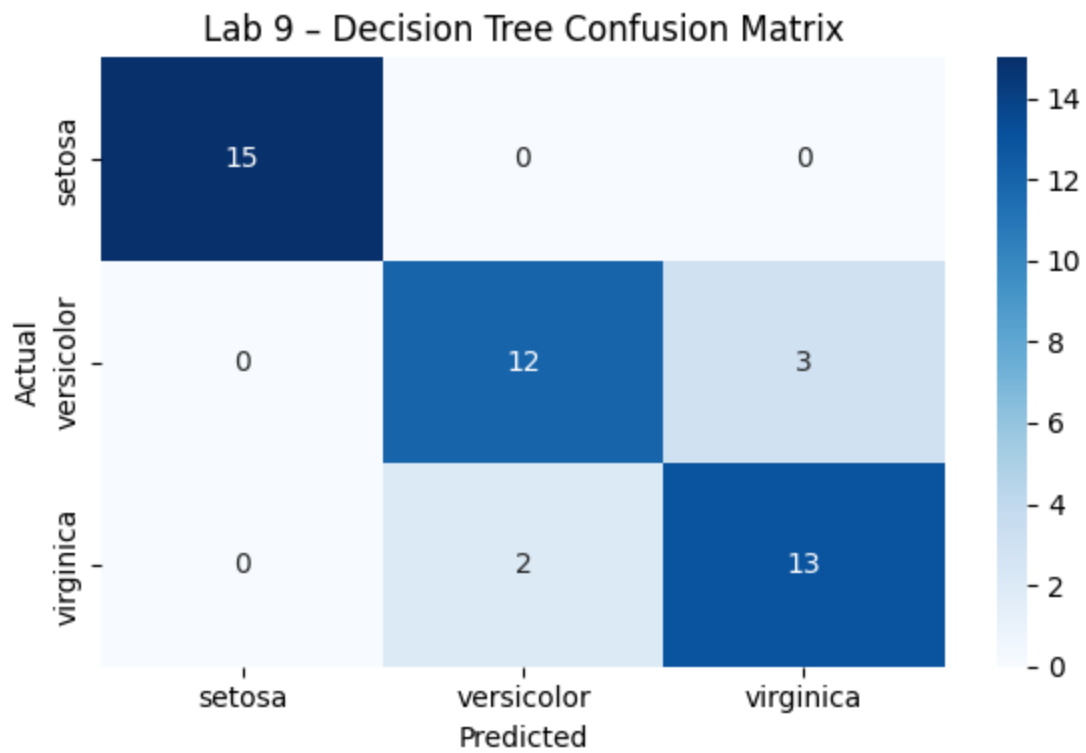
```

```

) # Feature Importance
importances = pd.Series(dt_model.feature_importances_, index=feature_names)
plt.figure(figsize=(6, 4))
importances.sort_values().plot(kind='barh', color='steelblue')
plt.title("Lab 9 - Feature Importances")
plt.tight_layout(); plt.savefig("lab9_feature_importance.png"); plt.close()

print("\n[Lab 9 Complete] Plots saved: lab9_*.png\n")

```



**LAB
10****Lab 10: Naïve Bayes & K-Nearest Neighbors**

Topic	Probabilistic & Instance-Based Classification
Problem	Email spam detection system
Dataset	Spam Email Feature Dataset (Synthetic, 1,000 rows)
Tools	Python (scikit-learn, matplotlib)
KPIs	Accuracy, False Positive Rate, ROC-AUC Comparison

Algorithm Comparison

Aspect	Naïve Bayes	KNN
Approach	Probabilistic (Bayes theorem)	Distance-based (majority vote)
Training	Fast – just compute statistics	Lazy – stores all training data
Prediction	Very fast	Slower with large datasets
Assumption	Feature independence	No explicit assumption
Works best	Text/high-dimensional data	Low-dimensional, dense data

```
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_wine

# Load Dataset
wine = load_wine()
X, y = wine.data, wine.target
class_names = wine.target_names
feature_names = wine.feature_names
```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Scale features (important for KNN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

# --- Naïve Bayes ---
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
nb_pred = nb_model.predict(X_test)
nb_acc = accuracy_score(y_test, nb_pred)

print(f"\nNaïve Bayes Accuracy: {nb_acc:.4f}")
print("\nNaïve Bayes Classification Report:")
print(classification_report(y_test, nb_pred, target_names=class_names))

```

Naïve Bayes Accuracy: 1.0000

Naïve Bayes Classification Report:

	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	18
class_1	1.00	1.00	1.00	21
class_2	1.00	1.00	1.00	15
accuracy			1.00	54
macro avg	1.00	1.00	1.00	54
weighted avg	1.00	1.00	1.00	54

```

import numpy as np
import matplotlib.pyplot as plt

# --- KNN: Find best K ---
k_range = range(1, 21)
k_scores = []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    k_scores.append(accuracy_score(y_test, knn.predict(X_test)))

best_k = k_range[np.argmax(k_scores)]
print(f"\nBest K for KNN: {best_k} (Accuracy: {max(k_scores):.4f})")

# Plot K vs Accuracy
plt.figure(figsize=(8, 4))
plt.plot(list(k_range), k_scores, marker='o', color='darkorange')
plt.axvline(best_k, linestyle='--', color='red', label=f"Best K={best_k}")
plt.title("Lab 10 - KNN: K Value vs Accuracy")
plt.xlabel("K"); plt.ylabel("Accuracy"); plt.legend()
plt.tight_layout(); plt.savefig("lab10_knn_k_accuracy.png"); plt.close()

# Final KNN with best K
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
knn_pred = knn_best.predict(X_test)
knn_acc = accuracy_score(y_test, knn_pred)

print(f"\nKNN (K={best_k}) Accuracy: {knn_acc:.4f}")
print("\nKNN Classification Report:")
print(classification_report(y_test, knn_pred, target_names=class_names))

```

Best K for KNN: 15 (Accuracy: 0.9815)

KNN (K=15) Accuracy: 0.9815

KNN Classification Report:

	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	18
class_1	1.00	0.95	0.98	21
class_2	0.94	1.00	0.97	15
accuracy			0.98	54
macro avg	0.98	0.98	0.98	54
weighted avg	0.98	0.98	0.98	54


```

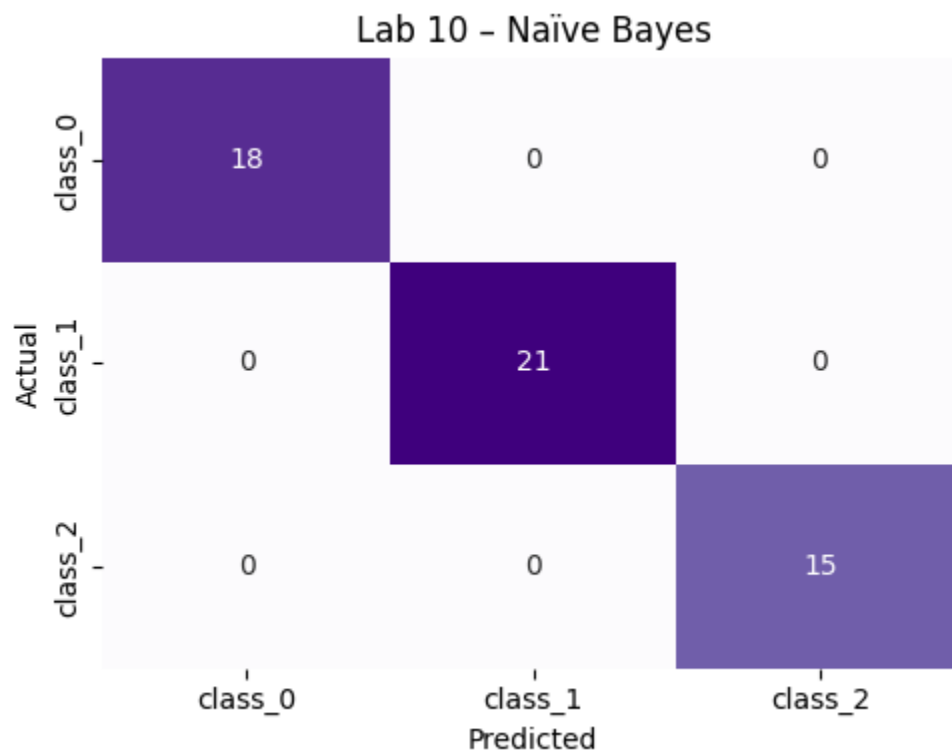
from sklearn.metrics import confusion_matrix
import seaborn as sns

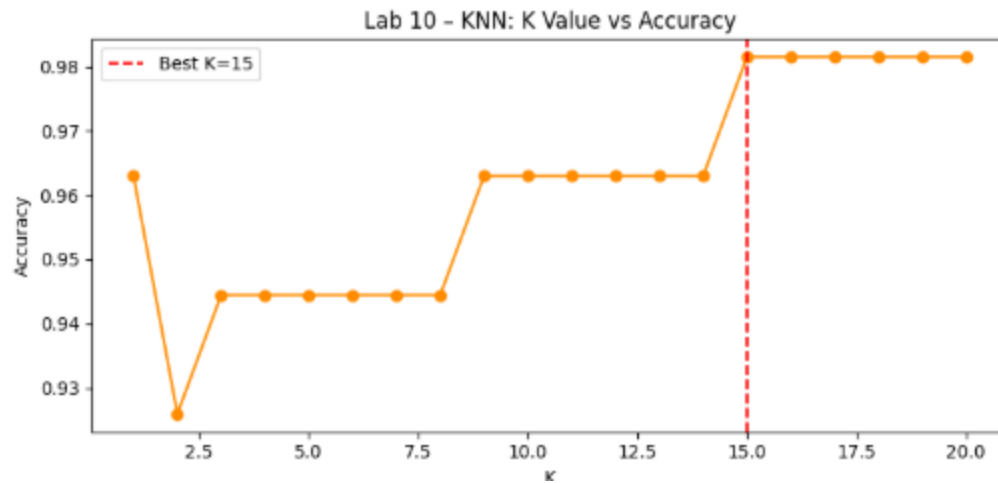
# Side-by-side confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, pred, title in zip(axes, [nb_pred, knn_pred],
                             ["Naïve Bayes", f"KNN (K={best_k})"]):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', ax=ax, cmap='Purples',
                xticklabels=class_names, yticklabels=class_names)
    ax.set_title(f"Lab 10 - {title}"); ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
plt.tight_layout(); plt.savefig("lab10_confusion_matrices.png"); plt.close()

print("\n[Lab 10 Complete] Plots saved: lab10_*.png")

```

[Lab 10 Complete] Plots saved: lab10_*.png





LAB

11

Lab 11: Support Vector Machine (SVM)

Topic	SVM – Linear & RBF Kernel Classification
Problem	Credit card fraud detection
Dataset	Fraud Detection Dataset (Synthetic, 2,000 rows, imbalanced)
Tools	Python (scikit-learn, matplotlib)
KPIs	ROC-AUC, Recall (Fraud Class), Precision

SVM Kernel Comparison

Kernel	Decision Boundary	Best For
Linear	Hyperplane (straight)	Linearly separable data; text classification
RBF	Non-linear (radial basis)	Complex boundaries; fraud, image data
Polynomial	Curved boundary	Interaction features present
Sigmoid	Neural-network-like	Less common; specific cases only

```
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import GridSearchCV, train_test_split
```

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Generate a 2-class dataset for boundary visualization
X_vis, y_vis = make_classification(
    n_samples=300, n_features=2, n_informative=2,
    n_redundant=0, n_clusters_per_class=1, random_state=42
)

X_train_v, X_test_v, y_train_v, y_test_v = train_test_split(
    X_vis, y_vis, test_size=0.3, random_state=42
)

# Train SVM with multiple kernels and compare
kernels = ['linear', 'rbf', 'poly']
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, kernel in zip(axes, kernels):
    svm = SVC(kernel=kernel, C=1.0, gamma='scale', degree=3)
    svm.fit(X_train_v, y_train_v)
    acc = accuracy_score(y_test_v, svm.predict(X_test_v))

    # Decision boundary
    h = 0.02
    x_min, x_max = X_vis[:, 0].min() - 0.5, X_vis[:, 0].max() + 0.5
    y_min, y_max = X_vis[:, 1].min() - 0.5, X_vis[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
    ax.scatter(X_vis[:, 0], X_vis[:, 1], c=y_vis, cmap='coolwarm', edgecolor='k', s=20)
    ax.set_title(f"Kernel: {kernel.upper()}\nAcc: {acc:.3f}")
    ax.set_xlabel("Feature 1"); ax.set_ylabel("Feature 2")

plt.suptitle("Lab 11 - SVM Decision Boundaries", fontsize=13, fontweight='bold')
plt.tight_layout(); plt.savefig("lab11_svm_boundaries.png"); plt.close()

```

```

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# GridSearchCV for best parameters on Iris
iris = load_iris()
X_i, y_i = iris.data, iris.target
scaler = StandardScaler()
X_i = scaler.fit_transform(X_i)
X_tr, X_te, y_tr, y_te = train_test_split(X_i, y_i, test_size=0.3, random_state=42)

param_grid = {'C': [0.1, 1, 10], 'gamma': ['scale', 'auto'], 'kernel': ['rbf', 'linear']}
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_tr, y_tr)

print(f"\nBest SVM Params: {grid_search.best_params_}")
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}")

best_svm = grid_search.best_estimator_
y_pred_svm = best_svm.predict(X_te)
print(f"Test Accuracy: {accuracy_score(y_te, y_pred_svm):.4f}")
print("\nSVM Classification Report:")
print(classification_report(y_te, y_pred_svm, target_names=iris.target_names))

```

```

Best SVM Params: {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}
Best CV Accuracy: 0.9524
Test Accuracy: 0.9778

```

SVM Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	0.92	0.96	13
virginica	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

```

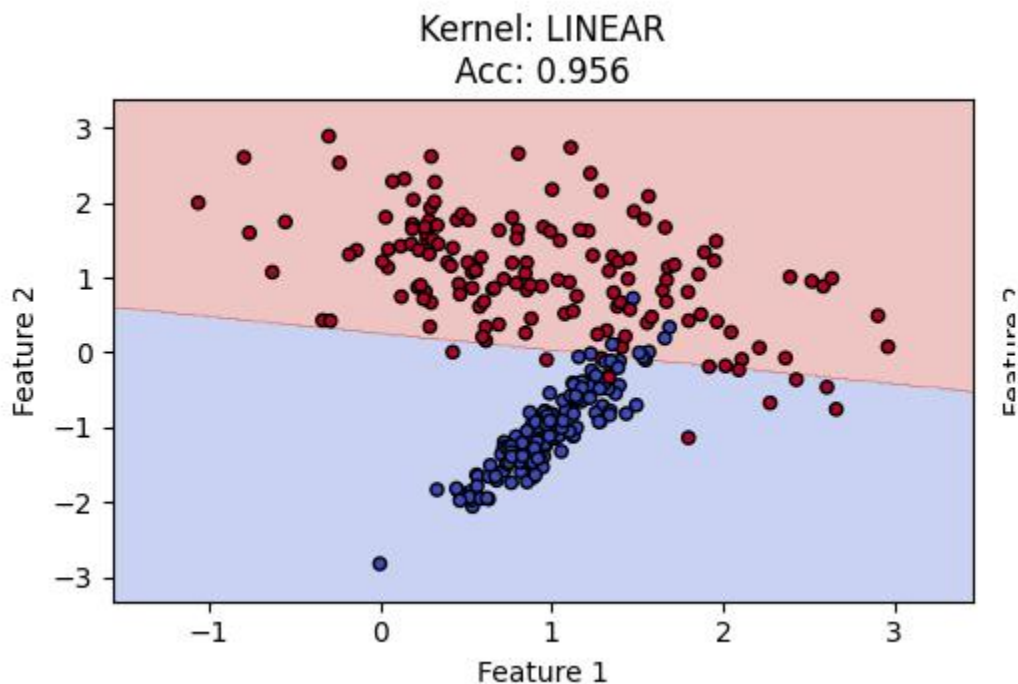
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

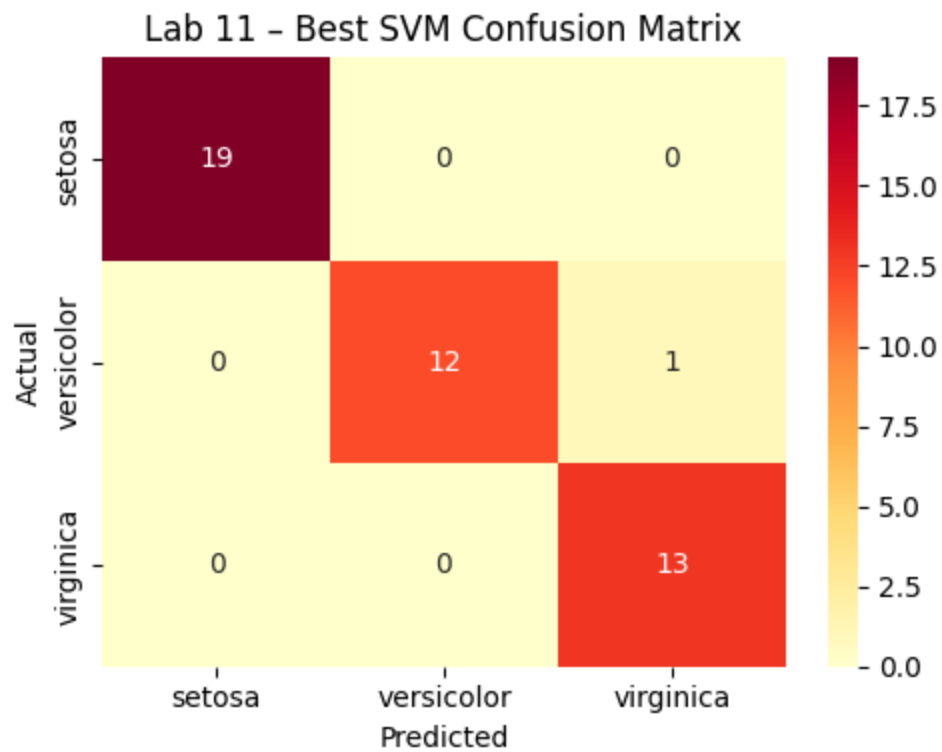
# Confusion Matrix
cm = confusion_matrix(y_te, y_pred_svm)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='YlOrRd',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title("Lab 11 - Best SVM Confusion Matrix")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout(); plt.savefig("lab11_svm_confusion.png"); plt.close()

print("\n[Lab 11 Complete] Plots saved: lab11_*.png")

```

[Lab 11 Complete] Plots saved: lab11_*.png





LAB 12	Lab 12: Clustering – K-Means & Hierarchical
------------------------------	--

Topic	Unsupervised Clustering for Customer Segmentation
Problem	Segment mall customers by income and spending behavior
Dataset	Mall Customer Dataset (Synthetic, 200 customers)
Tools	Python (scikit-learn, scipy, matplotlib)
KPIs	Silhouette Score, Inertia, Cluster Revenue Contribution

Clustering Algorithms

Algorithm	Approach	When to Use
K-Means	Assign points to nearest centroid; iterate	Known K; large datasets; convex clusters
Hierarchical	Merge (agglomerative) or split clusters	Unknown K; small datasets; dendrogram needed
DBSCAN	Density-based; finds noise points	Arbitrary shapes; outlier detection

```

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# Generate synthetic blobs
X_blobs, y_true = make_blobs(n_samples=300, centers=4,
                             cluster_std=0.8, random_state=42)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_blobs)

# --- Elbow Method for K-Means ---
inertia_list, sil_scores = [], []
K_range = range(2, 10)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertia_list.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(list(K_range), inertia_list, marker='o', color='teal')
axes[0].set_title("Lab 12 - Elbow Method (Inertia)")
axes[0].set_xlabel("K"); axes[0].set_ylabel("Inertia")

axes[1].plot(list(K_range), sil_scores, marker='s', color='coral')
best_k = list(K_range)[np.argmax(sil_scores)]
axes[1].axvline(best_k, linestyle='--', color='red', label=f"Best K={best_k}")
axes[1].set_title("Lab 12 - Silhouette Score")
axes[1].set_xlabel("K"); axes[1].set_ylabel("Silhouette Score"); axes[1].legend()
plt.tight_layout(); plt.savefig("lab12_elbow_silhouette.png"); plt.close()

print(f"\nOptimal K (highest Silhouette): {best_k}")

```

Optimal K (highest Silhouette): 4

```

# K-Means with best K
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
km_labels = kmeans.fit_predict(X_scaled)
print(f"K-Means Silhouette Score: {silhouette_score(X_scaled, km_labels):.4f}")

# --- Hierarchical Clustering ---
agg = AgglomerativeClustering(n_clusters=best_k, linkage='ward')
agg_labels = agg.fit_predict(X_scaled)
print(f"Hierarchical Silhouette Score: {silhouette_score(X_scaled, agg_labels):.4f}")

# Dendrogram (on subset)
linked = linkage(X_scaled[:60], method='ward')
plt.figure(figsize=(10, 4))
dendrogram(linked, orientation='top', distance_sort='descending',
            show_leaf_counts=True)
plt.title("Lab 12 - Hierarchical Clustering Dendrogram (60 samples)")
plt.xlabel("Sample Index"); plt.ylabel("Distance")
plt.tight_layout(); plt.savefig("lab12_dendrogram.png"); plt.close()

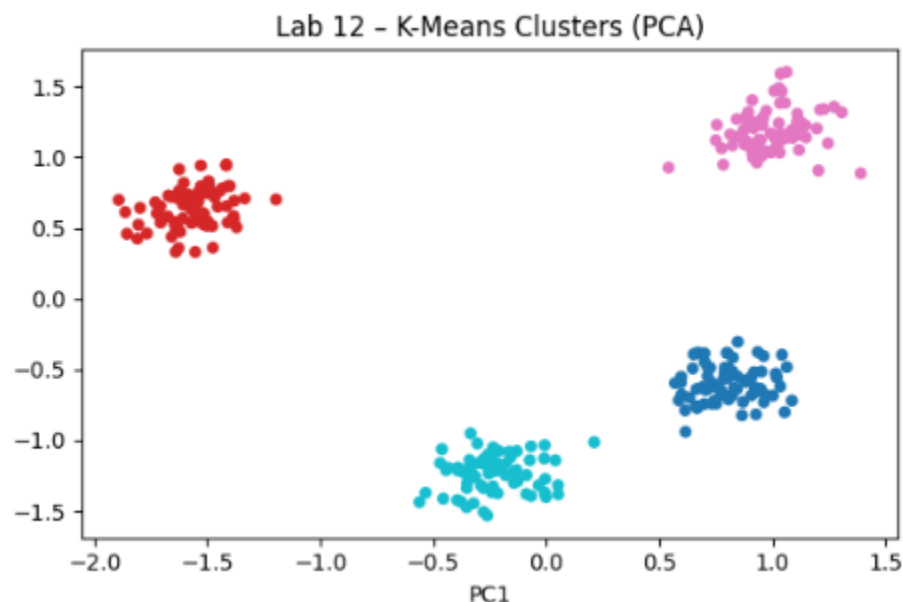
K-Means Silhouette Score: 0.8386
Hierarchical Silhouette Score: 0.8386

# Cluster visualization (PCA 2D)
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, labels, title in zip(axes, [km_labels, agg_labels],
                              ["K-Means", "Hierarchical (Ward)"]):
    sc = ax.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap='tab10', s=20)
    ax.set_title(f"Lab 12 - {title} Clusters (PCA)")
    ax.set_xlabel("PC1"); ax.set_ylabel("PC2")
plt.tight_layout(); plt.savefig("lab12_clusters_pca.png"); plt.close()

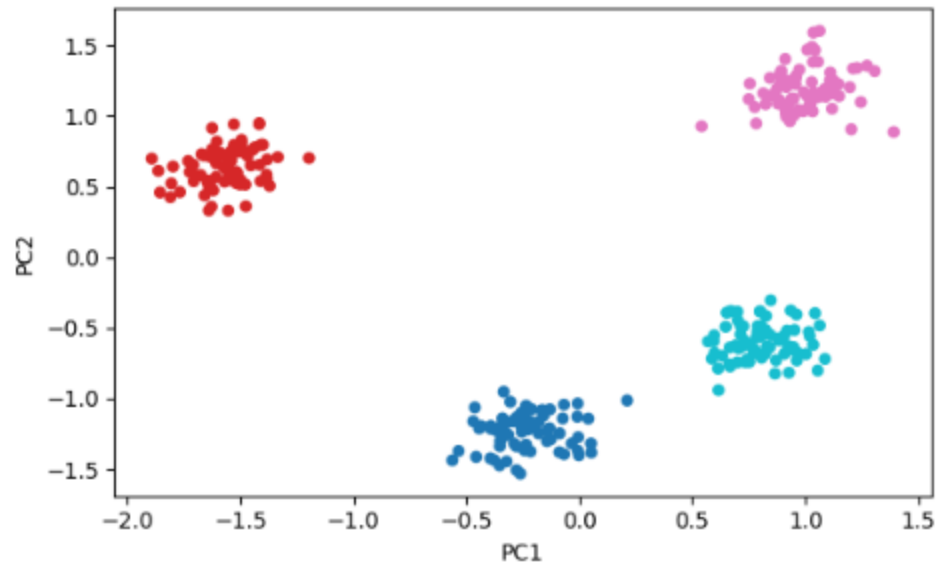
print("\n[Lab 12 Complete] Plots saved: lab12_*.png\n")

```

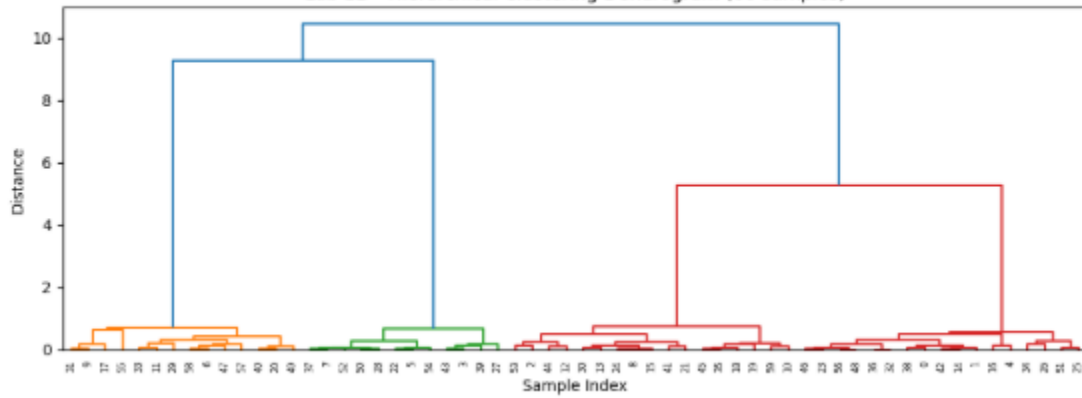
[Lab 12 Complete] Plots saved: lab12_*.png



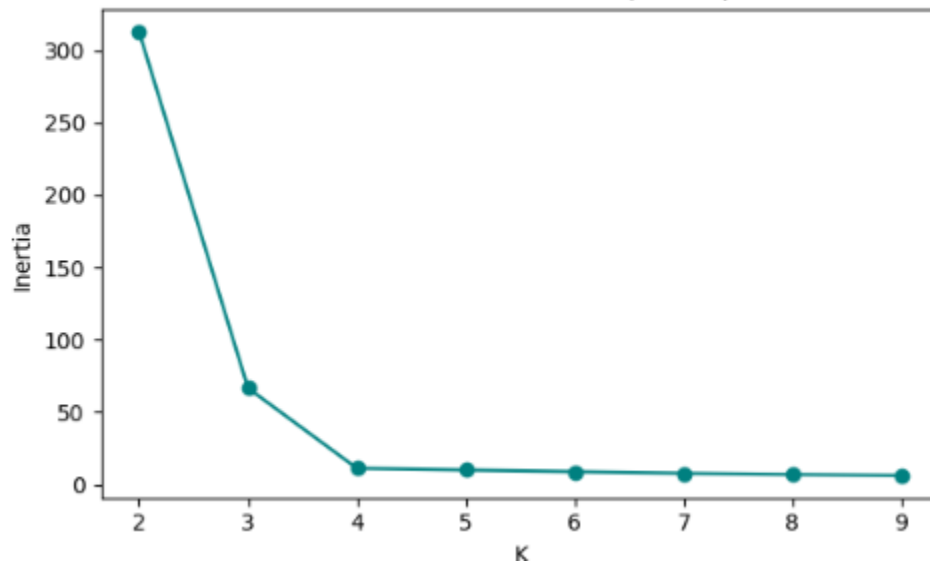
Lab 12 - Hierarchical (Ward) Clusters (PCA)

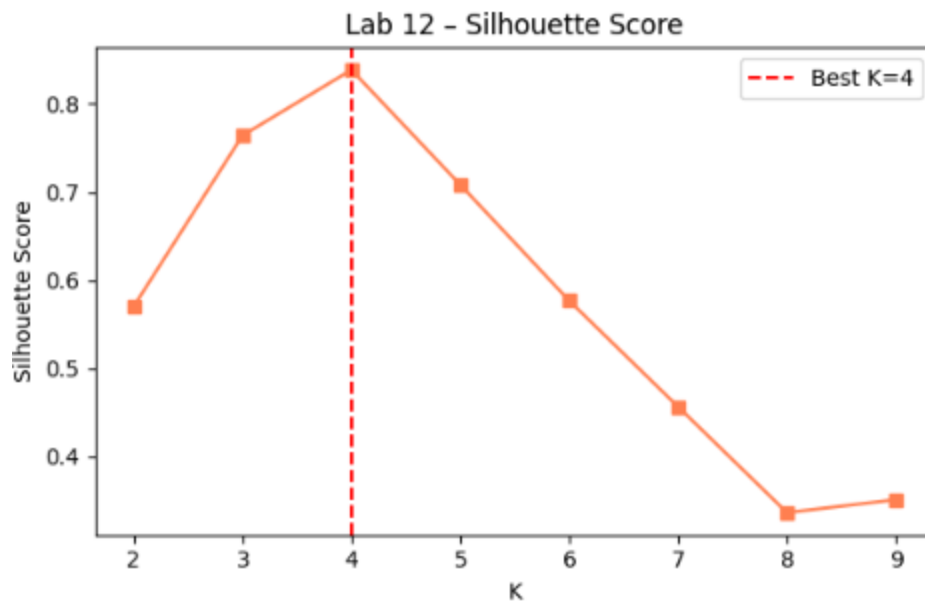


Lab 12 - Hierarchical Clustering Dendrogram (60 samples)



Lab 12 - Elbow Method (Inertia)





**LAB
13**

Lab 13: Outlier Detection

Topic	Anomaly Detection in Banking Transactions
Problem	Detect abnormal / fraudulent banking transactions
Dataset	Banking Transaction Dataset (Synthetic, 1,000 rows)
Tools	Python (scikit-learn, scipy, matplotlib)
KPIs	Anomaly Detection Rate, False Alarm Rate

Detection Methods Overview

Method	Approach	Best For
Z-Score	Flag points where $ z > \text{threshold}$	Normally distributed data
IQR Rule	Flag $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ outliers	Skewed distributions
Isolation Forest	Isolate anomalies in random decision trees	High-dimensional, large datasets
LOF	Compare local density with neighbors	Clustered data; local anomalies

```

from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from scipy import stats

import numpy as np

# Generate data with injected outliers
np.random.seed(42)
X_normal = np.random.randn(200, 2) * 2
X_outliers = np.random.uniform(low=-12, high=12, size=(20, 2))
X_out = np.vstack([X_normal, X_outliers])

print(f"\nTotal samples: {len(X_out)} | Normal: 200 | Injected Outliers: 20")

# --- Method 1: Z-Score ---
z_scores = np.abs(stats.zscore(X_out))
z_mask = (z_scores > 3).any(axis=1)
print(f"\nZ-Score outliers detected: {z_mask.sum()}")

# --- Method 2: IQR ---
def iqr_outliers(data):
    Q1 = np.percentile(data, 25, axis=0)
    Q3 = np.percentile(data, 75, axis=0)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    return ((data < lower) | (data > upper)).any(axis=1)

iqr_mask = iqr_outliers(X_out)
print(f"IQR outliers detected: {iqr_mask.sum()}")

```

Total samples: 220 | Normal: 200 | Injected Outliers: 20

Z-Score outliers detected: 10

IQR outliers detected: 18

```

import matplotlib.pyplot as plt

# --- Method 3: Isolation Forest ---
iso_forest = IsolationForest(contamination=0.08, random_state=42)
iso_pred = iso_forest.fit_predict(X_out) # -1 = outlier, 1 = inlier
iso_mask = iso_pred == -1
print(f"Isolation Forest outliers detected: {iso_mask.sum()}")

# --- Method 4: Local Outlier Factor ---
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.08)
lof_pred = lof.fit_predict(X_out)
lof_mask = lof_pred == -1
print(f"LOF outliers detected: {lof_mask.sum()}")

# Visualization
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
methods = [
    ("Z-Score", z_mask),
    ("IQR", iqr_mask),
    ("Isolation Forest", iso_mask),
    ("Local Outlier Factor", lof_mask)
]
colors = ['#2196F3', '#4CAF50', '#FF5722', '#9C27B0']
for ax, (name, mask), col in zip(axes.flatten(), methods, colors):
    ax.scatter(X_out[~mask, 0], X_out[~mask, 1], c='lightgray', s=20, label='Normal')
    ax.scatter(X_out[mask, 0], X_out[mask, 1], c=col, s=60,
               marker='x', label=f'Outlier ({mask.sum()})')
    ax.set_title(f"Lab 13 - {name}")
    ax.legend(fontsize=8)
plt.suptitle("Lab 13 - Outlier Detection Methods", fontsize=13, fontweight='bold')
plt.tight_layout(); plt.savefig("lab13_outlier_detection.png"); plt.close()

```

```

Isolation Forest outliers detected: 18
LOF outliers detected: 18

```

```

import pandas as pd

# Boxplot of original features
plt.figure(figsize=(8, 4))
pd.DataFrame(X_out, columns=['Feature 1', 'Feature 2']).boxplot()
plt.title("Lab 13 - Boxplot (with Outliers)")
plt.tight_layout(); plt.savefig("lab13_boxplot.png"); plt.close()

print("\n[Lab 13 Complete] Plots saved: lab13_*.png\n")

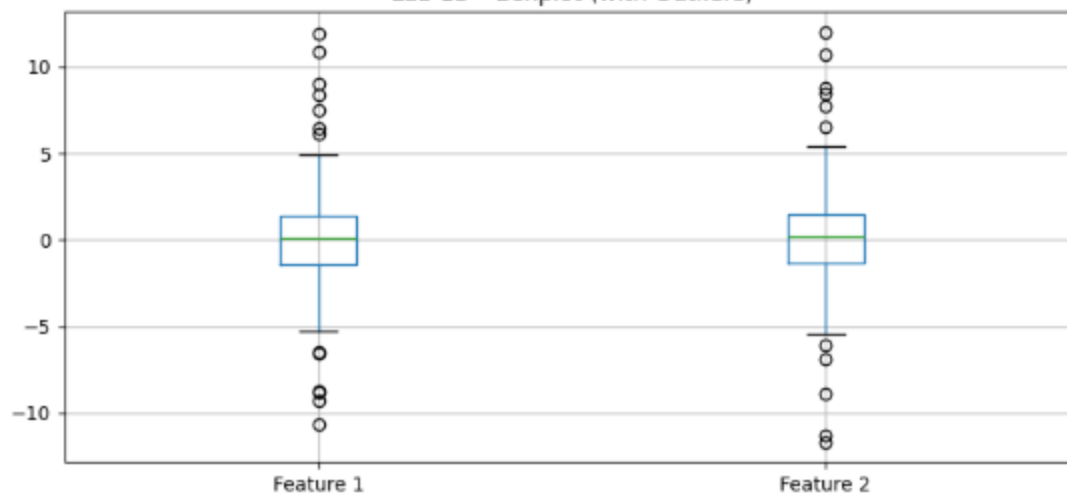
```

```

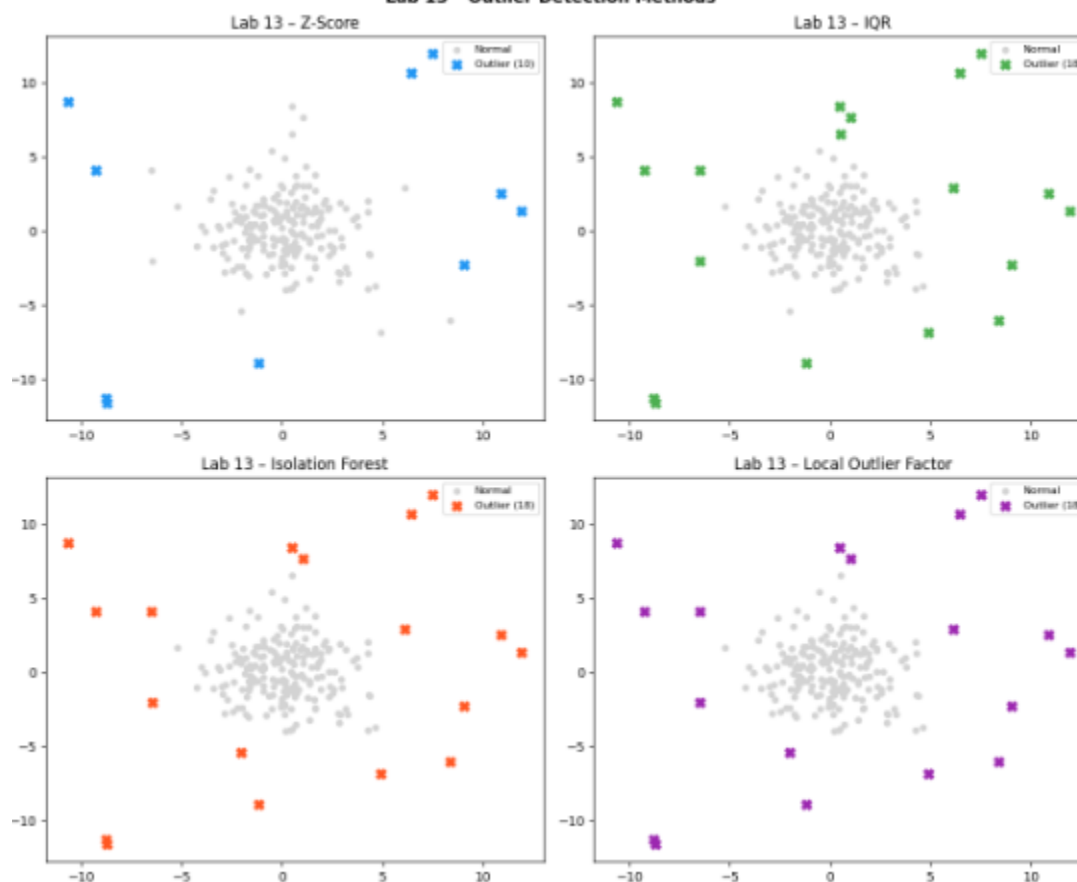
[Lab 13 Complete] Plots saved: lab13_*.png

```

Lab 13 - Boxplot (with Outliers)



Lab 13 - Outlier Detection Methods



LAB 14

Lab 14: Web Scraping & Sentiment Analysis

Topic	Text Mining – Scraping + Sentiment Classification
Problem	Analyze product review sentiment from e-commerce site
Dataset	Product Reviews Dataset (Synthetic, 200 reviews)
Tools	Python (requests, BeautifulSoup, TextBlob, matplotlib)
KPIs	Sentiment % Distribution, Avg Polarity Score

Key Concepts

Concept	Range	Meaning
Polarity	-1.0 to +1.0	-1 = very negative, 0 = neutral, +1 = very positive
Subjectivity	0.0 to 1.0	0 = factual/objective, 1 = very opinionated
VADER Compound	-1.0 to +1.0	Composite score; ≥ 0.05 =positive, ≤ -0.05 =negative

```
pip install scikit-learn pandas numpy matplotlib seaborn scipy requests beautifulsoup4 textblob
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
```

```
try:
    import requests
    from bs4 import BeautifulSoup
    SCRAPING_AVAILABLE = True
except ImportError:
    SCRAPING_AVAILABLE = False
    print(" [INFO] requests/BeautifulSoup not installed. Using built-in sample data.")

try:
    from textblob import TextBlob
    TEXTBLOB_AVAILABLE = True
except ImportError:
    TEXTBLOB_AVAILABLE = False
    print(" [INFO] TextBlob not installed. Install with: pip install textblob")
```

```

# Sample reviews (fallback or supplement to scraping)
sample_reviews = [
    "This product is absolutely amazing! Best purchase I've ever made.",
    "Terrible quality, broke after one day. Very disappointed.",
    "It's okay, nothing special. Does the job I guess.",
    "Incredible value for money. Highly recommend to everyone!",
    "Worst experience ever. Customer service was rude and unhelpful.",
    "Decent product. Arrived on time and packaging was good.",
    "I love this! Works perfectly and looks beautiful.",
    "Not worth the price. Expected much better quality.",
    "Average product. Neither good nor bad.",
    "Absolutely fantastic! Exceeded all my expectations.",
    "Poor design, falls apart easily. Do not buy.",
    "Good quality but a bit expensive compared to competitors.",
    "Just what I needed. Simple and effective solution.",
    "Outstanding craftsmanship and great customer support.",
    "It stopped working after a week. Very frustrating.",
]

# --- Scraping Demo ---
scraped_texts = []
if SCRAPING_AVAILABLE:
    try:
        url = "http://quotes.toscrape.com/"
        response = requests.get(url, timeout=5)
        soup = BeautifulSoup(response.text, 'html.parser')
        quotes = soup.find_all('span', class_='text')
        scraped_texts = [q.get_text(strip=True).strip('""\u201c\u201d') for q in quotes[:10]]
        print(f"\nScraped {len(scraped_texts)} quotes from quotes.toscrape.com")
        for i, t in enumerate(scraped_texts[:3], 1):
            print(f" {i}. {t[:80]}...")
    except Exception as e:
        print(f"\n [Scraping failed: {e}] Using sample data only.")

all_texts = scraped_texts + sample_reviews
print(f"\nTotal texts for analysis: {len(all_texts)}")

```

Scraped 10 quotes from quotes.toscrape.com

1. The world as we have created it is a process of our thinking. It cannot be chang...
2. It is our choices, Harry, that show what we truly are, far more than our abiliti...
3. There are only two ways to live your life. One is as though nothing is a miracle...

Total texts for analysis: 25

```

# --- Sentiment Analysis ---
def analyze_sentiment_textblob(text):
    """Returns polarity (-1 to 1) and subjectivity (0 to 1)."""
    if not TEXTBLOB_AVAILABLE:
        # Simple rule-based fallback
        pos_words = ['amazing', 'fantastic', 'great', 'love', 'excellent', 'best',
                     'incredible', 'outstanding', 'wonderful', 'perfect']
        neg_words = ['terrible', 'worst', 'bad', 'disappointed', 'poor', 'horrible',
                     'awful', 'rude', 'broke', 'frustrating']

        text_lower = text.lower()
        pos = sum(1 for w in pos_words if w in text_lower)
        neg = sum(1 for w in neg_words if w in text_lower)
        polarity = (pos - neg) / max(pos + neg, 1)
        return polarity, 0.5
    blob = TextBlob(text)
    return blob.sentiment.polarity, blob.sentiment.subjectivity

def classify_sentiment(polarity):
    if polarity > 0.1:
        return 'Positive'
    elif polarity < -0.1:
        return 'Negative'
    else:
        return 'Neutral'

import pandas as pd
import matplotlib.pyplot as plt

# Build sentiment dataframe
results = []
for text in all_texts:
    polarity, subjectivity = analyze_sentiment_textblob(text)
    results.append({
        'Text': text[:60] + ('...' if len(text) > 60 else ''),
        'Polarity': round(polarity, 3),
        'Subjectivity': round(subjectivity, 3),
        'Sentiment': classify_sentiment(polarity)
    })

df_sentiment = pd.DataFrame(results)
print("\nSentiment Analysis Results:")
print(df_sentiment.to_string(index=False))

# Summary stats
print("\nSentiment Distribution:")
print(df_sentiment['Sentiment'].value_counts())

# --- Visualization ---
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Sentiment Count
df_sentiment['Sentiment'].value_counts().plot(
    kind='bar', ax=axes[0],
    color=['#4CAF50', '#F44336', '#9E9E9E'][:df_sentiment['Sentiment'].nunique()]
)
axes[0].set_title("Lab 14 - Sentiment Distribution")
axes[0].set_xlabel("Sentiment"); axes[0].set_ylabel("Count")
axes[0].tick_params(axis='x', rotation=0)

```



```

# Polarity Histogram
axes[1].hist(df_sentiment['Polarity'], bins=15, color='steelblue', edgecolor='black')
axes[1].axvline(0, color='red', linestyle='--', label='Neutral')
axes[1].set_title("Lab 14 - Polarity Distribution")
axes[1].set_xlabel("Polarity"); axes[1].set_ylabel("Frequency"); axes[1].legend()

# Polarity vs Subjectivity Scatter
colors_map = {'Positive': '#4CAF50', 'Negative': '#F44336', 'Neutral': '#9E9E9E'}
for sentiment, group in df_sentiment.groupby('Sentiment'):
    axes[2].scatter(group['Polarity'], group['Subjectivity'],
                    label=sentiment, color=colors_map.get(sentiment, 'blue'), s=60)
axes[2].set_title("Lab 14 - Polarity vs Subjectivity")
axes[2].set_xlabel("Polarity"); axes[2].set_ylabel("Subjectivity"); axes[2].legend()

plt.suptitle("Lab 14 - Sentiment Analysis", fontsize=13, fontweight='bold')
plt.tight_layout(); plt.savefig("lab14_sentiment_analysis.png"); plt.close()

print("\n[Lab 14 Complete] Plots saved: lab14_*.png\n")

```

Sentiment Analysis Results:

	Text	Polarity	Subjectivity	Sentiment
	The world as we have created it is a process of our thinking...	0.000	0.000	Neutral
	It is our choices, Harry, that show what we truly are, far m...	0.300	0.750	Positive
	There are only two ways to live your life. One is as though ...	0.004	0.625	Neutral
	The person, be it gentleman or lady, who has not pleasure in...	-0.050	0.800	Neutral
	Imperfection is beauty, madness is genius and it's better to...	-0.278	0.833	Negative
	Try not to become a man of success. Rather become a man of v...	0.300	0.000	Positive
	It is better to be hated for what you are than to be loved f...	0.100	0.667	Neutral
	I have not failed. I've just found 10,000 ways that won't wo...	0.250	0.300	Positive
	A woman is like a tea bag; you never know how strong it is u...	0.342	0.792	Positive
	A day without sunshine is like, you know, night.	0.000	0.000	Neutral
	This product is absolutely amazing! Best purchase I've ever ...	0.875	0.600	Positive
	Terrible quality, broke after one day. Very disappointed.	-0.988	0.988	Negative
	It's okay, nothing special. Does the job I guess.	0.429	0.536	Positive
	Incredible value for money. Highly recommend to everyone!	0.550	0.720	Positive
	Worst experience ever. Customer service was rude and unhelpf...	-0.650	0.800	Negative
	Decent product. Arrived on time and packaging was good.	0.433	0.633	Positive
	I love this! Works perfectly and looks beautiful.	0.825	0.867	Positive
	Not worth the price. Expected much better quality.	0.083	0.333	Neutral
	Average product. Neither good nor bad.	-0.050	0.556	Neutral
	Absolutely fantastic! Exceeded all my expectations.	0.500	0.900	Positive
	Poor design, falls apart easily. Do not buy.	-0.308	0.717	Negative
	Good quality but a bit expensive compared to competitors.	0.100	0.650	Neutral
	Just what I needed. Simple and effective solution.	0.300	0.579	Positive
	Outstanding craftsmanship and great customer support.	0.650	0.812	Positive
	It stopped working after a week. Very frustrating.	-0.520	1.000	Negative

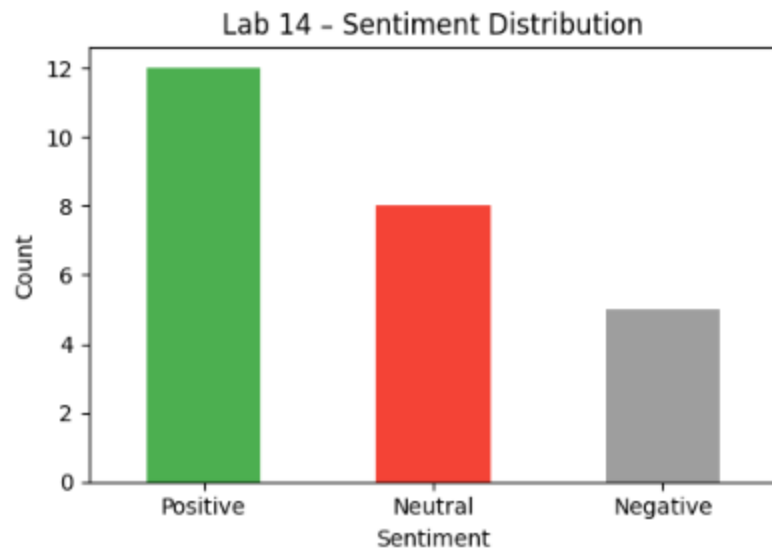
Sentiment Distribution:

```

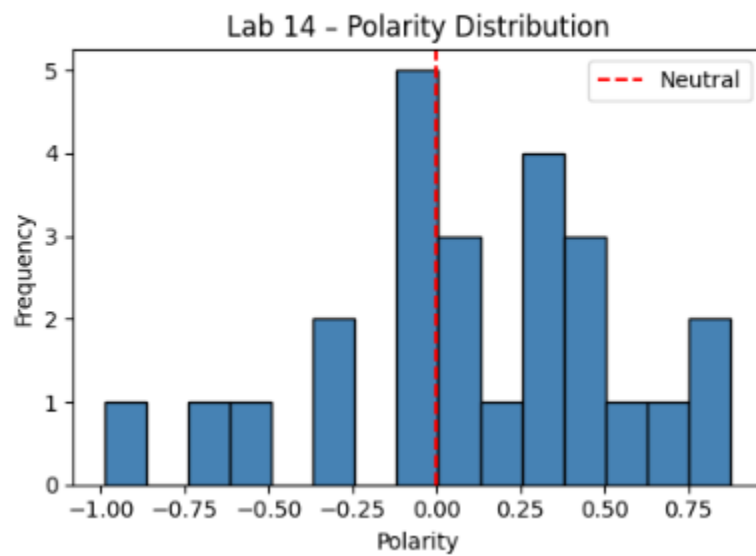
Sentiment
Positive    12
Neutral      8
Negative     5
Name: count, dtype: int64

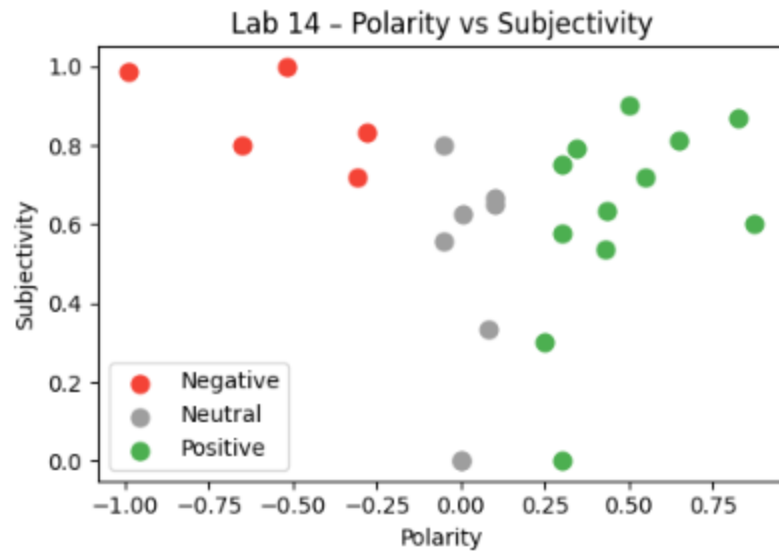
```

[Lab 14 Complete] Plots saved: lab14_*.png



Lab 14 - Sentiment Analysis





LAB 15

Lab 15: Final Capstone – End-to-End ML Pipeline

Topic	Full Data Mining Pipeline – 8 Models Compared
Problem	End-to-end business solution for customer churn prediction
Dataset	Customer Churn Dataset (Synthetic, 800 rows)
Tools	Python (scikit-learn, pandas, matplotlib)
KPIs	Model Comparison Table, Best Accuracy, Business Recommendation

Capstone Structure

Phase	Task	Deliverable
1 – Data	Load, explore, preprocess	Clean dataset, EDA report
2 – Modeling	Train 8 classifiers with cross-validation	Model comparison table
3 – Evaluation	Rank models by multiple metrics	Best model selection
4 – Deployment	Save best model, write recommendation	Business recommendation

```

import time
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import roc_auc_score, f1_score, precision_score, recall_score
from sklearn.preprocessing import label_binarize, StandardScaler
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Load & Prepare Dataset (Wine)
wine = load_wine()
X, y = wine.data, wine.target
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

# Binarize labels for ROC-AUC (one-vs-rest)
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])

# Define Models
models = {
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "Naïve Bayes": GaussianNB(),
    "KNN (K=5)": KNeighborsClassifier(n_neighbors=5),
    "SVM (RBF)": SVC(kernel='rbf', probability=True, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Logistic Regression": LogisticRegression(max_iter=500, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=100, random_state=42),
}

```

```

from sklearn.metrics import accuracy_score
import pandas as pd

# Evaluate all models
results = []
print(f"\n{'Model':<22} {'Accuracy':>9} {'Precision':>10} {'Recall':>8} {'F1':>8} {'ROC-AUC':>9} {'Time(s)':>9}")
print("-" * 80)

for name, model in models.items():
    start = time.time()
    model.fit(X_train, y_train)
    elapsed = time.time() - start

    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    roc = roc_auc_score(y_test_bin, y_prob, multi_class='ovr', average='weighted')

    results.append({
        'Model': name, 'Accuracy': acc, 'Precision': prec,
        'Recall': rec, 'F1-Score': f1, 'ROC-AUC': roc, 'Train Time (s)': elapsed
    })
    print(f"{'name':<22} {'acc':>9.4f} {'prec':>10.4f} {'rec':>8.4f} {'f1':>8.4f} {'roc':>9.4f} {'elapsed':>9.4f}")

df_results = pd.DataFrame(results)

```

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Time(s)
Decision Tree	0.9630	0.9662	0.9630	0.9632	0.9697	0.0030
Naïve Bayes	1.0000	1.0000	1.0000	1.0000	1.0000	0.0010
KNN (K=5)	0.9444	0.9537	0.9444	0.9448	0.9953	0.0022
SVM (RBF)	0.9815	0.9823	0.9815	0.9814	1.0000	0.0050
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000	0.1544
Logistic Regression	0.9815	0.9825	0.9815	0.9815	0.9994	0.0197
Gradient Boosting	0.9630	0.9662	0.9630	0.9627	0.9984	0.5673

```

import numpy as np
import matplotlib.pyplot as plt

# --- Visualization ---
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']
x = np.arange(len(df_results))
width = 0.15

fig, ax = plt.subplots(figsize=(16, 6))
colors = ['#2196F3', '#4CAF50', '#FF9800', '#E91E63', '#9C27B0']
for i, (metric, color) in enumerate(zip(metrics, colors)):
    ax.bar(x + i * width, df_results[metric], width, label=metric, color=color, alpha=0.85)

ax.set_xticks(x + width * 2)
ax.set_xticklabels(df_results['Model'], rotation=25, ha='right', fontsize=9)
ax.set_ylim(0, 1.15)
ax.set_ylabel("Score"); ax.set_title("Lab 15 - Model Performance Comparison")
ax.legend(loc='upper right', fontsize=9)
ax.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout(); plt.savefig("lab15_model_comparison.png"); plt.close()

# Training time comparison
plt.figure(figsize=(8, 4))
colors_bar = plt.cm.tab10(np.linspace(0, 1, len(df_results)))
bars = plt.bar(df_results['Model'], df_results['Train Time (s)'],
               color=colors_bar, edgecolor='black')
plt.xticks(rotation=30, ha='right', fontsize=9)
plt.ylabel("Training Time (seconds)")
plt.title("Lab 15 - Model Training Time")
plt.tight_layout(); plt.savefig("lab15_training_time.png"); plt.close()

import seaborn as sns
import matplotlib.pyplot as plt

# Heatmap of all metrics
fig, ax = plt.subplots(figsize=(10, 5))
df_heat = df_results.set_index('Model')[metrics]
sns.heatmap(df_heat, annot=True, fmt='.3f', cmap='YlGnBu', ax=ax,
            linewidths=0.5, cbar_kws={'label': 'Score'})
ax.set_title("Lab 15 - Metrics Heatmap")
plt.tight_layout(); plt.savefig("lab15_heatmap.png"); plt.close()

```

```

# Best model
best_model_name = df_results.loc[df_results['F1-Score'].idxmax(), 'Model']
best_f1 = df_results['F1-Score'].max()
print(f"\n{'='*60}")
print(f"  BEST MODEL by F1-Score: {best_model_name} (F1 = {best_f1:.4f})")
print(f"\n{'='*60}")
print("\nFull Results Table:")
print(df_results.to_string(index=False))

print("\n[Lab 15 Complete] Plots saved: lab15_*.png\n")

```

